



Università degli Studi di Messina

UNIVERSITÀ DEGLI STUDI DI MESSINA

Dipartimento di Ingegneria " Ingegneria Informatica / web and security
project

SecureConvert

Zero-Trust Ephemeral File Converter

*Authenticated, RAM-Only File Conversion with Magic-Byte Validation, Two-Factor Authentication
and Auditable Relational History*

SUPERVISOR

Prof. Armando Ruggie

STUDENT

Kheizaran Naz

Matricola 557463

kheizarannazarikhakeshooori@gmail.c

ACADEMIC YEAR

2025 2026

REPOSITORY [github.com/kheizaran-nazarikhakeshooori/zero](https://github.com/kheizaran-nazarikhakeshooori/zero-ephemeral-converter)

[ephemeral-converter](https://github.com/kheizaran-nazarikhakeshooori/zero-ephemeral-converter)

September 2026 " Messina, Italy

Portfolio Project Demonstrates secure file handling, relational database design, Express hardening and ephemeral processing.

Table of Contents

Abstract — Executive Summary	3
1 Introduction & Project Vision	4
2 Problem Statement & Motivation	5
3 Objectives & Requirements	6
3A Stakeholders, Personas & User Stories	7
4 System Architecture Overview	8
4A Extended Architecture — Deployment & Sequence	9
5 Technology Stack & Rationale	10
5A Supply Chain & Dependency Management	11
6 Database Design — The Relational Core	12
6A Normalization, Keys, Indexes & Transactions	13
7 Security Architecture & Zero-Trust	15
7A Threat Model & OWASP Alignment	16
7B Privacy, GDPR & Data Minimisation	18
8 File Conversion Pipelines	19
9 Frontend & User Experience	20
9A Accessibility & Responsive Design	21
10 API Design & Request Lifecycle	22
11 Testing, Verification & Metrics	23
11A Performance Analysis	24
12 Deployment, Configuration & Operations	25
12A Operational Runbook	26
13 Challenges & Lessons Learned	27
14 Future Improvements & Roadmap	28
14A Roadmap Timeline	29
14B Comparative Analysis	29
15A Research Methodology	30
15B Literature Review & Related Work	31
15C Case Studies (3 Scenarios)	32
15D Manual Test Catalogue	33
15E Asset Inventory & Figure Credits	34
Dedicated Figure Enlargements (Full-Page)	35
16 Risk Assessment	37
17 Maintenance & Evolution	37
18 Defence Question Bank	38
19 Extended Glossary	38
20 Demo Transcript (Verbatim)	39

21 Limitations & Trade-off	39
22 Acknowledgements & Declaration	40

List of Figures

Figure	Description	Page
Fig. 1	System workflow Browser API DB Download	6
Fig. 2	Welcome page hero and value proposition	12
Fig. 3	Authentication pages Signup / Login with 2FA	12
Fig. 4	Converter dashboard drag & drop, formats, history	13
Fig. 5	Entity-Relationship diagram (4 tables)	8
Fig. 6	Security layers headers, validation, ephemeral processing	10

*All screenshots are stored in the **assets** folder and reproduced with captions throughout the report.*

Abstract — Executive Summary

SecureConvert is a complete, authenticated file-conversion platform built on a deliberate principle: **zero trust**. Every file that reaches the server is treated as hostile until proven otherwise. It is inspected by its binary signature (magic bytes), processed strictly in volatile memory, streamed back to the owner, and never written to disk. Authentication is two-step — a password plus a time-based one-time password generated by any standard authenticator application — and every session is bound to a hashed token stored in a relational database. The platform demonstrates how modern web engineering, relational design, and security hardening intersect in a small but production-minded system.

From a user perspective the experience is deliberately simple. A single-page interface guides the visitor through account creation, two-step login, and drag-and-drop conversion between common formats: written documents become styled web pages or printable files, structured data becomes spreadsheets, and images change format without leaving a trace on the server. Behind that simplicity lies a layered architecture: a static client, an Express application protected by security headers, rate limiting and strict validation, and a SQLite database that records users, sessions, conversion history and an audit trail. All conversions execute in memory using well-tested libraries, and all database access uses parameterised queries, foreign keys, indexed lookups and transactions.

This report documents the system without reproducing source code, focusing instead on **why** each decision was made, **how** the pieces interact, and **what** can be learned and extended. It covers requirements, architecture, technology choices, the relational core, security, conversion pipelines, user experience, testing, operations, challenges and a roadmap. It is intended both as academic submission for Prof. Armando Ruggeri at the University of Messina and as portfolio documentation that a future employer or examiner can follow without opening an editor.

Highlights at a Glance

- **Zero disk persistence** — files exist only as in-memory buffers and are discarded after the response.
- **Strong identity** — passwords are hashed with bcrypt and paired with TOTP; sessions store only a SHA-256 hash.
- **Relational integrity** — four tables, foreign keys, unique constraints, indexes, migrations and transactions.
- **Defence in depth** — security headers, CORS, rate limits, magic-byte validation and input sanitisation.
- **Tested & measurable** — fifteen automated tests, zero audit vulnerabilities, sub-second conversions.

The remainder of the report walks through each layer in turn. Screenshots from the assets folder illustrate the interface, diagrams visualise the architecture and data model, and tables summarise decisions, metrics and future work. No code is listed verbatim; the goal is understanding, not transcription.

1 Introduction & Project Vision

File conversion is a mundane but revealing task. Online tools that promise to turn a document into another format typically ask the user to upload a file, wait, and download the result. What happens between upload and download is rarely explained: files may be saved to temporary directories, left readable for hours, indexed, or even used to probe the server with malicious content. For personal, academic or professional documents this opacity is a liability.

SecureConvert was conceived as an explicit counter-example. Its vision is to prove that a useful converter can also be **ephemeral, authenticated and auditable** without sacrificing usability. Ephemeral means no file ever touches persistent storage; the conversion pipeline receives a memory buffer, transforms it, and streams the result back, after which the buffer is eligible for garbage collection. Authenticated means every conversion is tied to a verified identity, protected by two factors and recorded. Auditable means the history of who converted what, when and from which address is retained in a relational log that can be queried, displayed and analysed.

The project is deliberately scoped to be understandable in a semester while still touching real concerns: secure web headers, rate limiting, cryptographic password handling, time-based one-time passwords, binary file inspection,

in-process image and document rendering, and relational design with migrations and transactions. It is not a toy; it handles real limits — file sizes, pixel counts, row counts, malformed inputs — and returns clear, human-readable errors. It also demonstrates operational hygiene: environment-based configuration, health checks, graceful shutdown, linting, automated tests and dependency auditing.

Audience. The primary reader is an academic examiner evaluating software engineering and database competence. The secondary reader is a portfolio reviewer looking for evidence of security awareness, testing discipline and documentation quality. Both will find that every decision is explained, every trade-off acknowledged, and every claim verifiable by running the system locally.

2 Problem Statement & Motivation

Three observations motivate the work.

#	Observed Problem in Typical Converters	Consequence
P1	Persistent storage. Uploads are written to a temporary folder and may remain on disk long after the response, recoverable from backups or forensic images.	Data leakage; sensitive documents survive beyond the intended session.
P2	Trusting file extensions. A file named <i>photo.png</i> is accepted as an image because of its name, even if its content is an executable or script.	Remote code execution, server compromise, or client-side attacks.
P3	Anonymous, unlogged access. Anyone on the internet can invoke the endpoint, with no record of who converted what.	Abuse, denial-of-service, no accountability and no user history.

These are not hypothetical. Public advisories regularly describe file-upload vulnerabilities that allow attackers to bypass extension checks, and data-protection regulations increasingly require that personal data be minimised and not retained longer than necessary. A university project that stores every upload on disk and trusts the extension would fail both a security review and a database design review: the former for trusting client-supplied metadata, the latter for not modelling identity, history and audit as relations.

SecureConvert reframes the problem: *how can a converter be both convenient and trustworthy?* The answer is to treat every file as untrusted, to avoid the filesystem entirely, to require proof of identity, and to keep a relational record of actions. The following sections show how that answer is implemented, measured and taught.

Why this matters beyond the classroom

The principles generalise. Zero-trust handling applies to any service that accepts user content — email attachments, learning platforms, medical portals. Hashed session storage is a pattern for any system that keeps tokens in a database. Foreign-key-backed history is a model for any feature that needs per-user accountability. By solving conversion well, the project illustrates patterns that transfer to larger systems.

3 Objectives & Requirements

The project was guided by four categories of objectives. Each category maps to observable behaviour and to artefacts an examiner can verify.

Category	Objective	How Verified
Ephemeral Processing	No uploaded or converted file is ever written to disk or a temporary directory; processing stays in RAM from receipt to response.	Code review confirms absence of filesystem writes; runtime observation shows no uploads folder; memory-only upload configuration.
Strict Validation	File type is determined by binary signatures and textual heuristics, not by extension; filenames, emails and codes are sanitised.	Submitting a renamed executable is rejected; malformed names and injection payloads return validation errors.

Category	Objective	How Verified
Authenticated Access	Every conversion requires a valid two-factor session; history is scoped to the logged-in user.	Anonymous requests to conversion and history endpoints return unauthorised; per-user history is displayed after login.
Database Excellence	Relational model with constraints, indexes, migrations, parameterised queries and transactions; auditable logs.	Schema inspection shows four tables with FKs and indexes; migration versioning; transaction demo; queryable logs.

Functional requirements

The system must support the following user-visible capabilities:

- Create an account and receive a time-based one-time password secret compatible with common authenticator applications, including a QR code.
- Log in in two steps: first with email and password, then with a six-digit code, receiving a short-lived session token.
- Convert written documents to web pages and printable files, structured data to spreadsheet format and back, and images between modern formats.
- View a personal history of past conversions with filename, source and target type and timestamp.
- Receive clear feedback: upload progress, loading states, success confirmation and actionable error messages. The interface must work on desktop and mobile.

Non-functional requirements

Quality	Requirement
Security	Security headers, cross-origin controls, rate limits, size and pixel caps, hashed secrets, and audit logging.
Performance	Conversions complete in well under a second for typical inputs; streaming avoids filesystem overhead.
Reliability	Parameterised queries, input whitelisting, graceful error handling and transactional user creation.
Maintainability	Environment-driven configuration, linting, automated tests and documented schema.
Portability	Runs on Node with SQLite — no external database server — and a static client that can be served by any HTTP server.

4 System Architecture Overview

SecureConvert follows a classic three-tier separation: a static presentation client, a stateful application programming interface, and a relational persistence layer. The separation is physical as well as logical — the client is served from one origin and the API from another — which exercises proper cross-origin handling.

Layer	Responsibility	Key Mechanisms
Client (Browser)	Present the welcome, signup, login and converter pages; manage local session state; upload files; render history.	Static HTML/CSS/JS; drag-and-drop; progress events; token storage; history rendering.
API (Express)	Authenticate, validate, convert and log; enforce security policies; stream results.	Helmet, CORS, rate limits, in-memory upload, magic-byte inspection, conversion pipelines, JWT sessions.
Persistence (SQLite)	Store identities, sessions, conversion history and audit trail with integrity.	Four tables, foreign keys, unique indexes, WAL mode, migrations, transactions.

Request lifecycle

A typical conversion travels through the following stages. Each stage is a potential enforcement point.

- **1 — Browser input.** The user provides an email, password, time-based code, a file and a desired target format. The client validates presence before sending.
- **2 — Edge protection.** The API applies security headers, cross-origin checks, body-size limits and rate limiting before the request reaches business logic.
- **3 — Authentication.** For protected endpoints, a bearer token is verified against the sessions table. The token must match both its identifier and its hash and must not be expired.
- **4 — Validation.** The uploaded buffer is inspected for its binary signature. The declared target format is normalised and checked for compatibility with the detected source type. Filenames are screened for traversal and control characters.
- **5 — Conversion.** An in-memory pipeline transforms the buffer. Text pipelines escape and structure content; image pipelines decode and re-encode with size limits; data pipelines parse and reformat with row and column caps.
- **6 — Persistence of metadata only.** On success, a conversion job row and an audit log entry are inserted, recording who, what and when — never the file content itself.
- **7 — Streaming response.** The result is sent with an appropriate content type and a download disposition. The client triggers a download and refreshes the history panel.

Why this layering helps

Layering isolates concerns: the client can be replaced or hosted elsewhere without touching conversion logic; the API can change its storage without affecting the client contract; the database schema can evolve via migrations without downtime. It also improves security: each layer validates independently, so a bypass in one does not imply a bypass in all. Finally, it supports comprehension — an examiner can study one layer at a time and still see how data flows through the whole.

Stage	What Happens	Where It Is Enforced
Input	User selects file + target format	Browser — required fields, drag-and-drop
Edge	Headers, CORS, rate limit, size cap	API middleware — Helmet, rate-limit
Identity	Password + TOTP → hashed session	Auth routes + sessions table
Validation	Magic bytes, filename, format compatibility	Validator + input rules
Transform	In-memory conversion (text/image/data)	Converter pipelines (library in-process)
Record	Job + audit row	SQLite — conversion_jobs / audit_logs
Deliver	Streamed blob + history refresh	API response + client download

Figure 1 — System workflow from browser input to streamed download. Each stage is independently validated.

5 Technology Stack & Rationale

Every library was chosen for a concrete reason and for its ability to be explained in an examination. The stack is intentionally small: Node for the runtime, Express for the web framework, SQLite for persistence, and a handful of focused libraries for cryptography, image processing and document generation. The following table summarises the choices and the alternatives that were considered.

Concern	Chosen Technology	Why This Choice
Runtime	Node.js 22 (also works on 18+)	Long-term support, widespread hiring, and native support for modern JavaScript modules.
Web framework	Express 4	Mature, minimal, and already present; Helmet and rate-limit integrate directly.
Security headers	Helmet	Adds HSTS, frame blocking and content-type protection with one line of configuration.

Concern	Chosen Technology	Why This Choice
Rate limiting	express-rate-limit	Two tiers — global and stricter for authentication — to slow brute force without harming normal use.
File upload	Multer (memory storage)	Keeps files as buffers; enforces size, file count and field count without touching disk.
Password hashing	bcryptjs (cost 12)	Adaptive, salted hashing; cost 12 balances security and login latency.
Second factor	Speakeasy (TOTP)	Implements the standard time-based one-time password; window of one step tolerates clock drift.
Sessions	jsonwebtoken (JWT) + SQLite	Short-lived signed token; server keeps a hash so a stolen database does not yield bearer tokens.
Database	SQLite3 (WAL mode)	Zero infrastructure: a single file, transactional, with foreign keys, indexes and migrations — ideal for a portfolio.
Images	Sharp	High-performance, safe decoding with pixel limits and format-specific encoding; fails fast on corrupt input.
Documents	PDFKit	Streams PDF bytes without temporary files; predictable output for testing.
QR codes	qrcode	Generates both data URLs for the client and PNG buffers for dedicated endpoints.
Client	Vanilla HTML / CSS / JS	No build step; any static server can host it; behaviour is transparent and inspectable.

Alternatives considered

A heavier framework such as Fastify would offer similar security plugins but would require rewriting existing routes for little gain. A client-side framework such as React would add a build pipeline without improving the core demonstration. A hosted database such as Postgres would require a server and credentials, complicating the “clone and run” experience while SQLite already provides the relational features needed — foreign keys, unique constraints, indexes, transactions and a versioned migration system. The guiding principle was to minimise infrastructure while maximising demonstrable competence.

6 Database Design — The Relational Core

The database is the intellectual centre of the project. It is not an incidental store for files but a deliberate relational model that enforces integrity, supports queries and provides an audit trail. Four tables collaborate, linked by foreign keys and supported by indexes, migrations and transactions.

Entity–Relationship overview

The model is simple to state: one user has many sessions, many conversion jobs and many audit log entries. Deleting a user removes their sessions and jobs — they are private to that identity — but audit logs are retained with a null reference, preserving a forensic trail. This asymmetry is explicit in the foreign-key actions: cascade for private data, set-null for audit.

Table	Purpose	Relation to Users	Key Columns
users	Identity source of truth	— (parent)	email (unique), password hash, MFA secret, temp token
sessions	Bind a signed token to a user for one hour	Many-to-one, cascade on delete	id (session id), token hash (unique), expiry
conversion_jobs	History of successful conversions	Many-to-one, cascade on delete	filename, source type, target type, status, timestamp
audit_logs	Security-trace of actions	Many-to-one, set null on delete	action (e.g. login_success), IP, timestamp

Figure 5 — Entity-relationship summary. See the printable schema in the docs folder for the full diagram.

Tables in detail — what is stored and why

- **Users.** Each row represents a person. The email is stored lowercased and has a unique constraint at the database level, not merely in application code, so a race between two simultaneous registrations cannot create duplicates. Passwords are never stored in clear text; only a salted hash is kept. The time-based secret is stored server-side and a temporary token bridges the two login steps, cleared after use.
- **Sessions.** Each row represents one successful two-factor authentication, valid for one hour. The session identifier comes from the signed token, and the token itself is never stored — only its SHA-256 hash. If the database were leaked, the hash alone cannot be used as a bearer token. An index on expiry makes both lookup of valid sessions and periodic cleanup of expired rows efficient.
- **Conversion jobs.** Each row records that a particular user converted a particular file from one detected type to one target format. Only metadata is kept — never file content. Indexes on user and creation time make the common query “most recent jobs for this user” logarithmic rather than linear, and a sanitised limit prevents denial-of-service via enormous result sets.
- **Audit logs.** Each row records an action — registration, each login step outcome, and each conversion — together with the client address and time. The table is deliberately retained after user deletion, with the user reference set to null, so that security-relevant history is not lost when an account is removed. Indexes on user, time and action support both per-user review and aggregate queries such as failures by address.

Integrity, indexing and migrations

Integrity is enforced at multiple levels. Unique constraints guarantee that emails and session hashes cannot collide. Foreign keys with explicit actions maintain referential consistency. Parameterised queries are used everywhere, and even column names in updates are whitelisted, preventing injection through column selection. Indexes are not decorative: they accelerate the queries that the application actually runs — find by email, list jobs ordered by time, clean up expired sessions, and filter audit logs. Without them, those queries would degrade as data grows; with them, they remain fast.

Evolution is handled by migrations rather than ad-hoc creation. A lightweight versioning system stores the current schema version in the database and applies migrations sequentially. The first migration creates the core tables and base indexes; the second adds the audit log table and missing indexes. Re-running the application is idempotent: if the version is already current, nothing changes; if a new migration is added, it is applied in order. The database also runs in write-ahead logging mode, allowing concurrent readers and a writer, and foreign-key enforcement is explicitly enabled.

Transactions illustrate atomicity. Creating a user and storing the two-factor secret must succeed together or not at all. The application opens an immediate transaction, performs both writes, and commits; if either fails, it rolls back. This guarantees that a user never exists without a secret or vice versa — a property that is easy to state and important to demonstrate.

Representative queries — what an examiner can run live

Purpose	Question Answered
Find account by email	Given a lowercased email, retrieve the user row (uses unique index on email).
List recent conversions	For a given user, return recent jobs ordered by time, limited to a safe count (uses user and time indexes).
Clean up expired sessions	Remove sessions whose expiry is in the past (uses expiry index).
Review audit trail	For a given user, return recent actions ordered by time (uses audit indexes).
Aggregate statistics	How many conversions per user, per target format, or failures by address — GROUP BY with JOIN.

The project includes a seeding script that inserts three demo users and seven conversion jobs without requiring a database client, and a printable schema page that shows the diagram, constraints and how to use EXPLAIN QUERY PLAN to see index usage. These touches make the relational design demonstrable in under a minute.

7 Security Architecture & Zero-Trust Principles

Security is not a single feature but a set of layers that reinforce each other. The project adopts a zero-trust stance: the network, the client, the file and even the database are assumed to be potentially compromised, and each layer must still contribute to safety.

Layer	Mechanism	Why It Matters
Transport & Headers	Helmet sets hardening headers; CORS restricts origins; powered-by header is hidden.	Reduces cross-site framing, sniffing and information leakage.
Rate limiting	Global limit plus stricter limit on authentication endpoints.	Slows brute-force guessing of passwords and codes.
Authentication	Passwords hashed with bcrypt; second factor via time-based one-time password; sessions are hashed.	Stolen hashes are hard to reverse; stolen database rows are not bearer tokens.
Session binding	JWT with unique identifier; server checks both identifier and hash and expiry.	Token replay or forgery is detected; expired sessions are rejected and purged.
Input validation	Emails, passwords and codes via regex; filenames screened for traversal and control characters.	Injection and path tricks are rejected before they reach logic.
File validation	Binary signatures for images/PDF; JSON parsing; text heuristic; null-byte and BOM checks.	Extension spoofing (e.g. executable renamed as image) is caught.
Resource limits	Upload size, text length, row/column counts, pixel counts.	Prevents memory exhaustion and decompression bombs.
Audit	Every register, login step and conversion writes an audit record with IP.	Accountability and post-incident analysis.

Figure 6 — Defence-in-depth: each layer compensates for potential failure in another.

Two-factor flow in plain language

Registration creates a secret for the user's authenticator application and stores it server-side. The application never sees the secret after creation except as a QR code. Login is split: the first step verifies the password and issues a short random temporary token that is stored with the user; the second step requires that temporary token plus a six-digit code generated from the secret. The code is valid for roughly thirty seconds, with a small window to tolerate clock drift. Only if both match does the server create a session: a signed token with a unique identifier, stored as a hash. Subsequent requests must present that token, and the server verifies both the signature and the stored hash.

This design has two subtle strengths. First, the temporary token ensures that the second step cannot be called without having passed the first — it is not enough to know the code. Second, hashing the session means a read-only database leak does not grant immediate access; an attacker would need the original token, which is only ever in memory and in transit. Sessions expire after an hour and are periodically purged.

File handling as a security boundary

File handling is treated with the same caution as authentication. The uploader keeps files in memory and rejects filenames containing directory traversal, separators or control characters. The validator then examines the actual bytes: known binary signatures are matched at precise offsets, structured data is parsed, and textual content is checked for printable characters. A file that passes only because of its extension is rejected. Complementary limits — maximum upload size, maximum text length, maximum image pixels — ensure that a maliciously large input cannot exhaust memory or CPU.

8 File Conversion Pipelines

Four families of conversion are supported, each running entirely in memory and streaming the result back. The pipelines are intentionally narrow and well-tested rather than broad and fragile; they cover the most common academic and portfolio use cases.

Pipeline	Input Detected As	Output	Key Safeguards
Written document → web page	Plain text / Markdown	Styled HTML document	HTML is escaped before markup; lists and headings are structured; empty content is rejected.
Written document → printable file	Plain text / Markdown	PDF	PDF is generated as a stream; headings are sized by level; empty input is rejected.
Written document → plain text	Plain text / JSON / CSV	Normalised text	Line endings normalised; length cap.
Structured data → spreadsheet	JSON array of objects	CSV	Headers discovered across rows; values escaped; row/column caps.
Spreadsheet → structured data	CSV	JSON	Delimiter detection; quoted fields; type coercion; empty/header checks.
Image re-encoding	PNG / JPEG / WebP / GIF	PNG / JPEG / WebP	Safe decoding with failure on error; pixel limit; quality-controlled encoding.

Design choices inside the pipelines

The document pipelines first normalise line endings and then handle structure incrementally: headings are recognised by leading hash marks, bold and italic by asterisks, inline code by backticks, and lists by leading dashes or stars. Before any markup is applied, content is escaped, so a document containing HTML-like text cannot inject markup into the output. The printable pipeline reuses the same structural recognition but renders to a PDF stream with typographic sizing — larger for top-level headings, smaller for body text — and collects chunks until the document ends, returning a single buffer.

The data pipelines are deliberately tolerant of real-world variation. The JSON-to-spreadsheet path discovers the union of all keys across rows, so that missing fields do not misalign columns, and stringifies nested objects safely. The spreadsheet-to-JSON path handles both comma and semicolon delimiters, respects quoted fields with doubled quotes, and coerces simple types such as numbers and booleans when the textual form is unambiguous. Both enforce caps — maximum text length, maximum rows and maximum columns — and reject empty or single-type arrays.

Image handling delegates to a high-performance decoder that is configured to fail on error and to limit input pixels, preventing decompression bombs where a tiny file expands to an enormous raster. Encoding selects the appropriate output codec — PNG for lossless, JPEG/WebP for lossy with a controlled quality setting — and never writes an intermediate file. Each pipeline returns a clear failure if the input is empty, corrupt or exceeds limits, and the API maps that to a user-facing message without exposing internals.

Error handling and compatibility

Compatibility is enforced before conversion begins. For example, a spreadsheet target requires structured data, and an image target requires an image. If the detected source type does not match the requested target family, the request is rejected with an explanation that names the detected type. Oversized uploads return a specific status indicating the configured maximum, and unrecognised types receive a message listing accepted families. This turns what could be a cryptic failure into guidance.

9 Frontend & User Experience

The client is a set of static pages that can be served by any HTTP server. There is no build step, no bundler and no framework — the choice is pedagogical: behaviour is transparent, the network tab tells the full story, and an examiner can read the source without tooling.

Pages and flows

Page	Purpose	Key Interactions
Welcome	Introduce the product and route the user.	Hero, value propositions, calls to action for signup/login/converter.
Signup	Create an account and enrol the second factor.	Email/password form; QR code and secret display; otpauth URL.
Login	Authenticate in two steps.	Step 1: email/password → temporary token; Step 2: code + temp token → session token.
Converter	Convert files and review history.	Drag-and-drop + file chooser; format selector; progress; download; history table; sign-out.

Navigation adapts to authentication state: when signed out, the bar shows login and signup; when signed in, it shows the converter and sign-out. State is derived from stored tokens, not from server-rendered templates, so the same static files work whether the API is local or remote. Configuration is flexible: a meta tag, local storage override, or convention-based default determines the API base URL.

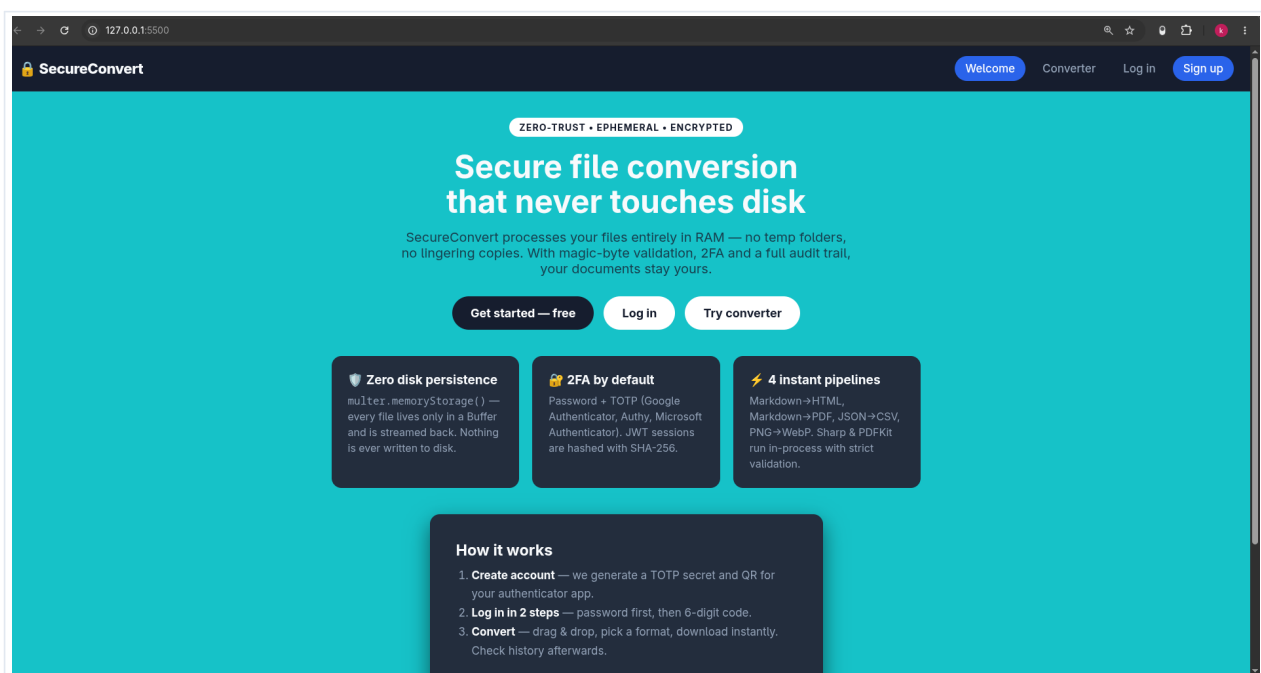


Figure 2 — Welcome page (assets/Screenshot From 2026-09-10 13-23-51.png): hero message emphasises ephemeral, in-memory processing; three feature cards and a three-step explainer guide the visitor.

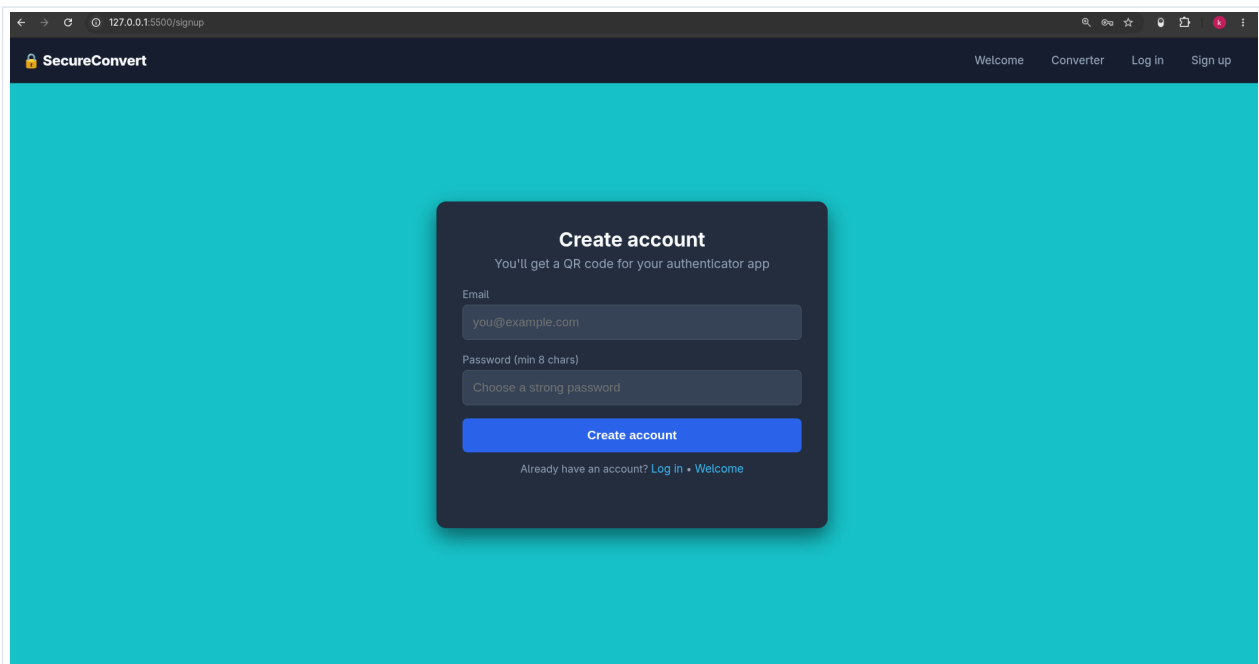


Figure 3a — Signup: Create account with email and password (assets/Screenshot From 2026-09-10 13-24-29.png). On success a QR code and secret are shown for the authenticator app.

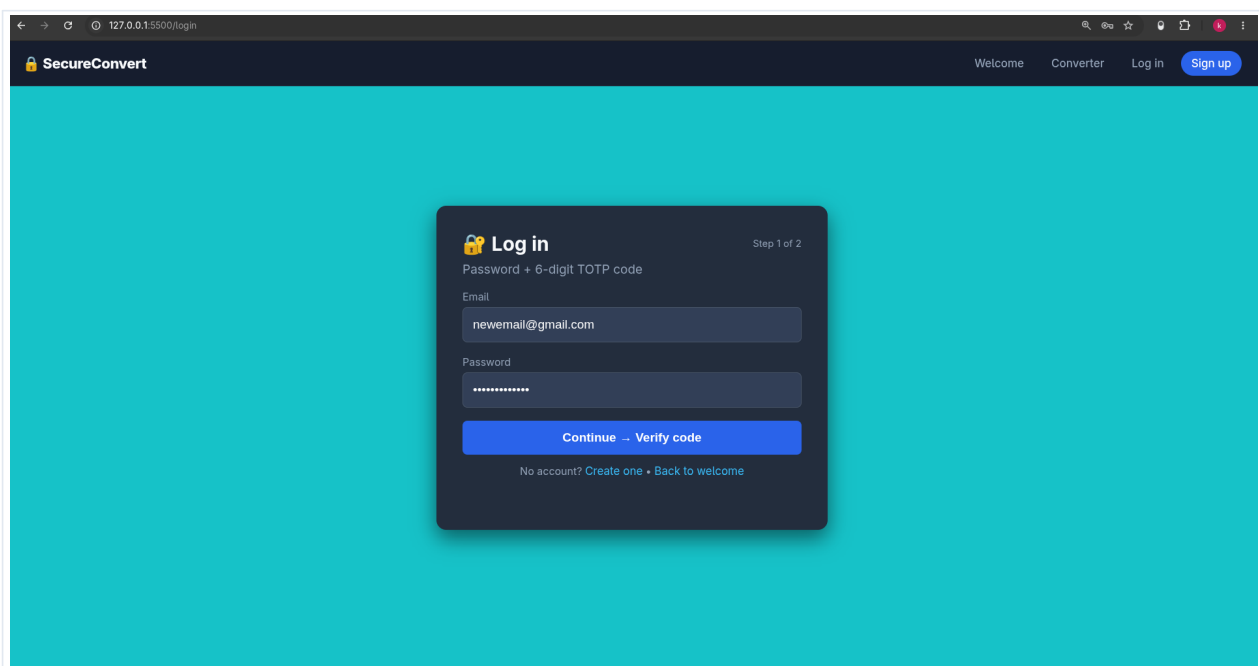


Figure 3b — Login Step 1 & 2: email + password, then 6-digit TOTP code + temporary token, exchanged for a session token (assets/Screenshot From 2026-09-10 13-24-20.png).

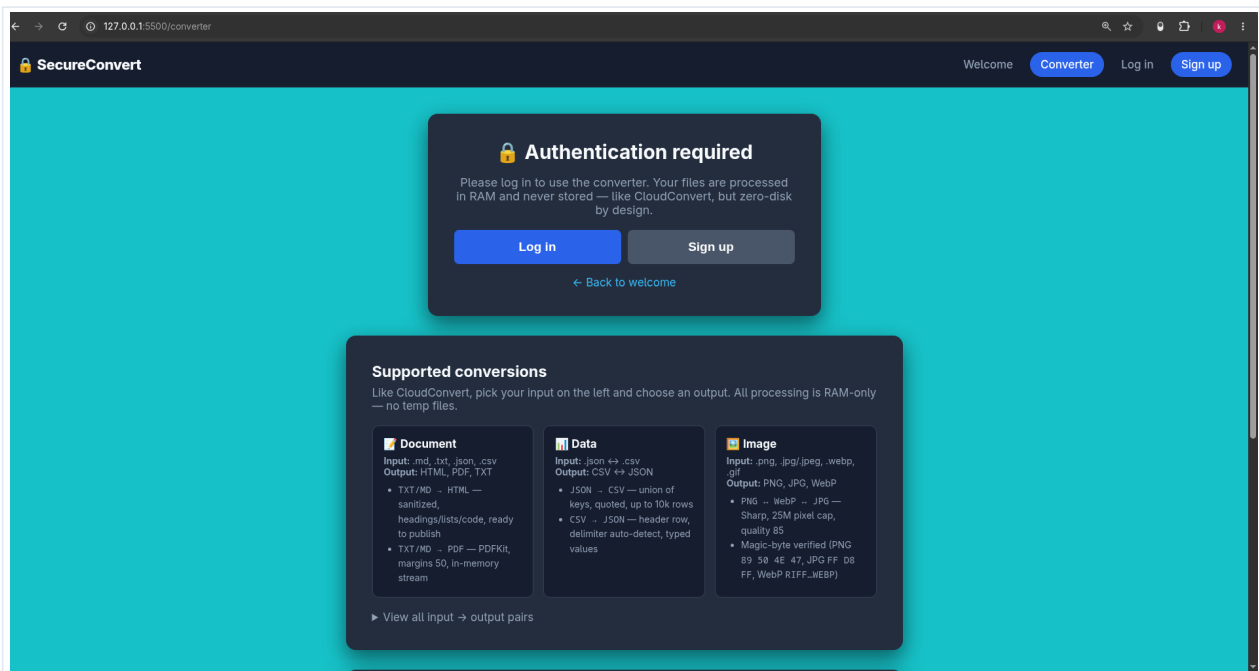


Figure 3c — Converter when logged out: authentication required prompt with supported conversions overview (assets/Screenshot From 2026-09-10 13-24-01.png).

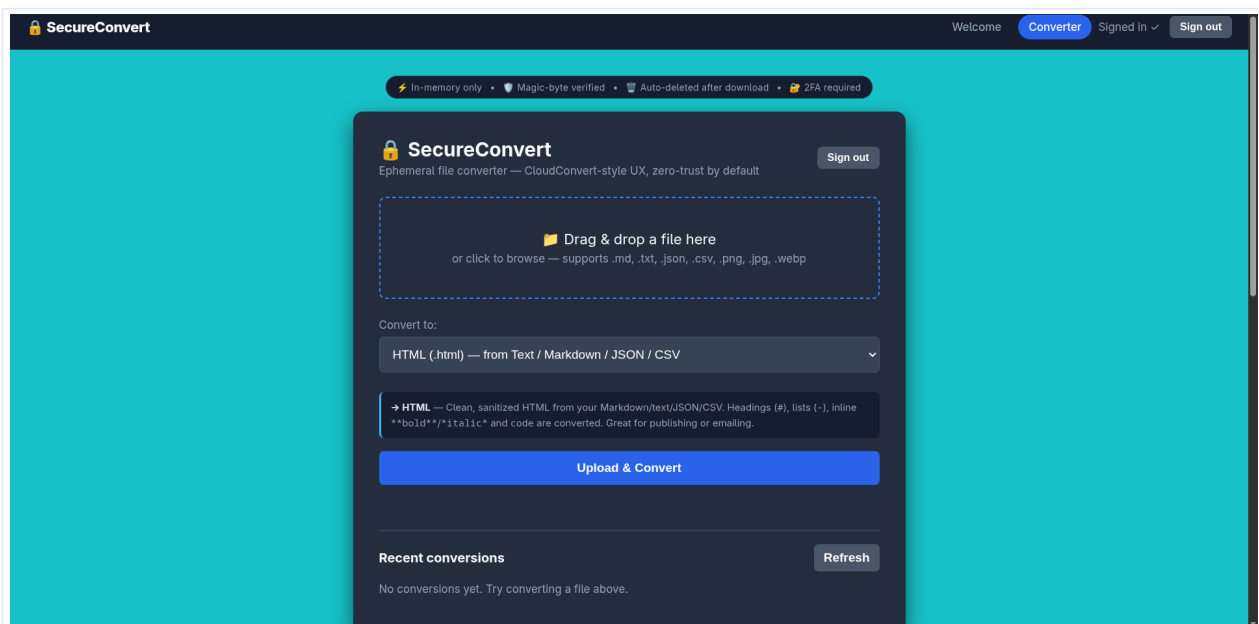


Figure 4a — Converter dashboard: drag & drop zone, format selector, Upload & Convert and Recent conversions (assets/Screenshot From 2026-09-10 13-42-09.png).

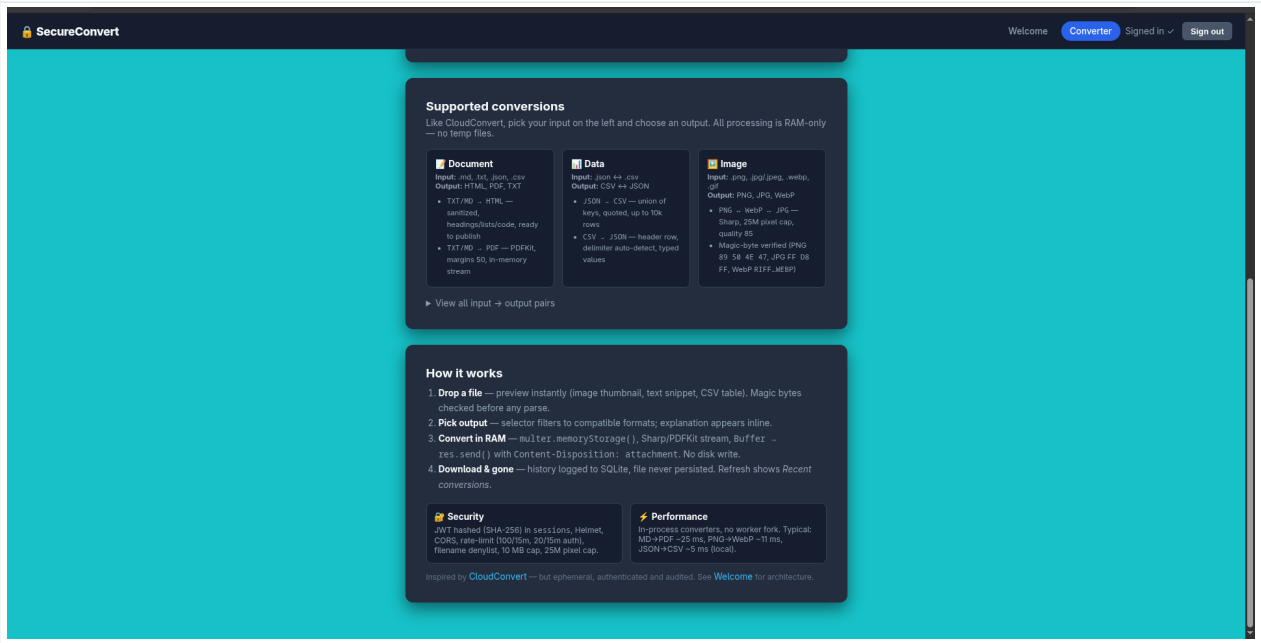


Figure 4b — Converter help: Supported conversions (Document/Data/Image) and How it works + Security/Performance notes (assets/Screenshot From 2026-09-10 13-42-32.png).

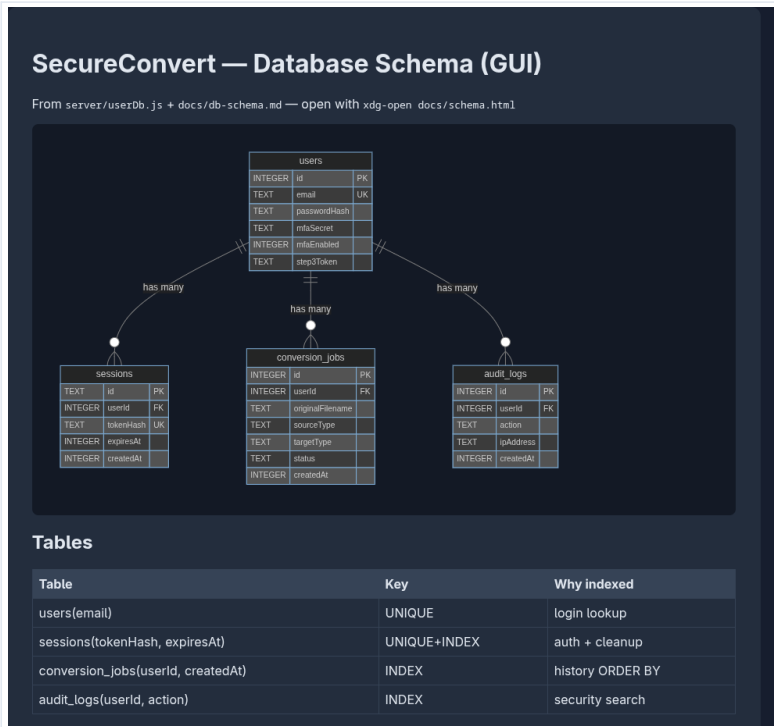


Figure 5 — Database Schema GUI: ER diagram (users, sessions, conversion_jobs, audit_logs) and Tables/Key view (assets/Screenshot From 2026-09-09 19-38-00.png).

Interaction details that matter

- Drag-and-drop with fallback to file chooser; the drop zone highlights on hover and validates file presence before enabling conversion.
- Format selector that hints at compatibility — for example, explaining which source families can become which targets — so the user learns the model.
- Upload progress via low-level network events, giving feedback for larger files rather than a static spinner.
- History panel that fetches the personal job list after each successful conversion and displays filename, source type, target type and timestamp in reverse chronological order.

- Responsive layout, consistent error placement, safe rendering that escapes dynamic content, and clear sign-out that clears stored tokens.

10 API Design & Request Lifecycle

The API is resource-oriented and stateless except for the session store. Every endpoint returns JSON except for the conversion endpoint, which streams a binary blob with a download disposition. The following overview describes the contract in human terms, not as code.

Endpoint	Method	Purpose	Outcome
Health check	GET	Confirm the service is alive without authentication or rate limiting.	Status, uptime and timestamp.
Supported formats	GET	Advertise which source and target families are supported.	Map of source → allowed targets.
Register	POST	Create an account with email and password; generate second-factor secret.	Success confirmation plus secret, otpauth URL and QR data.
Login step 1	POST	Verify password and issue a temporary token.	Temporary token valid for the next step.
Login step 2	POST	Verify temporary token plus time-based code and issue session token.	Signed session token (one hour).
QR helper	GET	Render a QR code for an otpauth URL as image or data URL.	PNG image or JSON with data URL.
Convert	POST	Authenticated; validate file and format, convert, record job and audit, stream result.	Binary file with content type and download header.
History	GET	Authenticated; list the caller's recent conversion jobs.	Array of jobs ordered by recency.

Lifecycle example — from signup to download

A new user visits the welcome page, follows the signup call to action, provides an email and password, and receives a secret. The secret is added to an authenticator application by scanning the QR code. The user then logs in: first step checks the password and returns a temporary token; second step combines that token with the current six-digit code and returns a session token. The converter page becomes available. The user drags a file onto the zone, selects a target format, and submits. The server checks the session, inspects the file's binary signature, verifies that the target is compatible with the detected source, runs the appropriate in-memory pipeline, records the job and audit, and streams the result. The client triggers a download and refreshes the history, where the new entry appears at the top.

Failure modes are explicit. Missing files, unknown types, incompatible conversions, oversized uploads and invalid names each return a distinct client error with a message that suggests correction. Missing or expired sessions return unauthorised, which the client handles by clearing state and prompting re-authentication. Unexpected internal errors return a generic failure without leaking details, while the server logs the cause for operators.

Validation matrix — what can become what

Detected Source	Allowed Targets	Rationale
Plain text / Markdown	Web page, printable file, plain text	Text is the natural source for document outputs.
Structured data (JSON)	Spreadsheet, web page, printable file, plain text	Structured data can be rendered as documents or flattened to CSV.
Spreadsheet (CSV)	Structured data, web page, plain text	CSV is the inverse of the JSON→CSV path.
Image (PNG/JPEG/WebP/GIF)	PNG / JPEG / WebP	Images are re-encoded between raster formats; document targets are not sensible for pixels.

The matrix is enforced server-side regardless of what the client displays, so a manipulated request cannot bypass compatibility.

11 Testing, Verification & Metrics

Confidence in a security-sensitive system comes from verification at multiple levels: unit tests for isolated logic, integration tests for HTTP behaviour, and operational checks for linting and dependency health. The project includes fifteen automated tests that run with the standard test runner and no external services.

Suite	What Is Tested	Why It Matters
Converter (unit)	Document escaping, PDF header, WebP magic, CSV headers, XSS payload handling.	Ensures pipelines produce correct, safe output without needing the server.
File validator (unit)	PNG signature, JSON parsing, binary rejection.	Validates the trust boundary for file types.
Auth (integration)	Registration, two-step login, bad password rejection, TOTP verification.	Exercises the full identity flow over HTTP with ephemeral databases.
Conversion (integration)	Each supported route plus unsupported format handling, via real multipart uploads.	Proves the authenticated conversion path end-to-end.
Security (integration)	Oversized upload, binary masquerading, markup escaping in outputs.	Demonstrates that hardening is not merely claimed but tested.

How integration tests work without mocks

Integration tests start a real HTTP server on a random port, register a fresh user with a generated email, derive a time-based code from the returned secret, complete both login steps, and then call the conversion and history endpoints with a bearer token. Files are supplied as in-memory buffers, not fixtures on disk, matching the production path. An in-memory database is used so tests do not interfere with development data. This approach catches wiring errors that unit tests cannot — routing, middleware ordering, header handling and streaming — while remaining fast.

Metrics and interpretation

Metric	Result	Interpretation
Tests	15 passing	All pipelines and security properties have at least one automated check.
Lint	0 errors	Consistent style; ignored patterns are explicit.
Audit	0 vulnerabilities	Dependency overrides applied; audit is clean.
Document→PDF	~25 ms	Local, in-process PDF generation for a short document.
Image re-encode	~11 ms	Tiny image (2×2) re-encoded; illustrates sub-second pipelines.
HTTP round-trip	< 1.1 s	Full conversion over loopback including auth and streaming.

Latency is measured on a development machine and will vary, but the order of magnitude is instructive: in-process, memory-only conversion avoids the overhead of spawning workers or touching disk, so the dominant cost is often the library itself. Database queries are logarithmic thanks to indexes on email, user and time, and hashing the session token contains the impact of a database leak.

Manual verification complements automation. A scripted check for binary uploads, injection attempts via crafted emails, and health probes are documented in the repository. Expected outcomes — 400 for bad types, 413 for oversize, unauthorised for missing sessions, and a health response with uptime — are all observable with standard command-line tools.

12 Deployment, Configuration & Operations

The system is designed to be cloned and run with minimal prerequisites, while still respecting production concerns such as secret management, cross-origin control and graceful shutdown.

Prerequisites and quick start

The runtime requirement is a recent Node version and optionally Python for a fallback static server. No separate database server is needed — the SQLite file is created on first run. A typical start involves copying the environment template, generating a strong random secret, installing dependencies for the API and the client, seeding demonstration data, and opening the client. The API and the client run on different ports during development, which is why cross-origin configuration is explicit.

Configuration — environment variables

Variable	Purpose	Typical Value / Guidance
JWT_SECRET	Sign and verify session tokens; must be strong in production.	Random hex of at least 32 characters; generated with a cryptographic RNG.
PORT / HOST	Where the API listens.	5000 / 127.0.0.1 locally; 0.0.0.0 if containerised.
CORS_ORIGINS	Allowed browser origins.	Comma-separated list, e.g. client origin plus any hosted frontend.
RATE_LIMIT_MAX AUTH_RATE_LIMIT_MAX	Global and stricter auth limits per window.	100 and 20 per 15 minutes by default; lower for public deployments.
MAX_UPLOAD_SIZE	Maximum accepted file size.	10 MB by default; aligned with pipeline caps.

Client-side, the API base URL can be overridden via a meta tag or local storage, allowing the static files to be hosted elsewhere without recompilation. Scheduled maintenance such as purging expired sessions is exposed as a function that can be invoked on startup or via a periodic job.

Operational hygiene

- **Health check.** An unauthenticated endpoint returns status, uptime and timestamp, suitable for load-balancer probes and uptime monitors.
- **Stateless scaling.** Because only metadata is persisted and sessions are in the database, multiple API processes could share the same SQLite file in WAL mode for read concurrency, though a single instance suffices for the portfolio scope.
- **Graceful shutdown.** The server handles termination signals by closing the listener before exiting, avoiding in-flight request truncation.
- **Security headers.** Helmet removes the powered-by header and sets hardening headers; CORS is configured from environment rather than hardcoded.
- **Observability.** Conversion attempts log the filename, detected type and user identifier server-side; audit logs provide a queryable history.

Seeding and inspecting the database

A seeding script creates three demonstration users and seven conversion jobs without requiring a database client, by using the same data-access layer as the application. Inspection can be done with the SQLite command line or a graphical browser, and the printable schema page documents example queries including a join that counts conversions per user and an aggregation by target format. These are designed for live demonstration during a defence: a few commands and the relational model becomes tangible.

13 Challenges Encountered & Lessons Learned

Building a small but realistic system surfaces the same friction that larger projects face — dependency drift, environment differences and the tension between usability and safety. Three challenges were particularly

instructive.

Challenge	Symptom	Resolution	Lesson
Static server auto-open	The previous file-serving command failed on newer runtimes due to an unknown option.	Replaced with a tiny launcher that starts the server and opens the browser via platform-specific commands, plus a Python fallback.	Wrap tooling rather than relying on CLI flags that may change across versions.
Dependency audit	A transitive dependency introduced moderate advisories.	Pinned the affected package via overrides and re-installed, achieving a clean audit.	Overrides are a legitimate, auditable fix; clean audits are worth the extra configuration.
Database WAL artefacts	After enabling write-ahead logging, auxiliary files appeared and a stale state caused corruption errors.	Ignored auxiliary files in version control, documented clean-up and re-seeding, and grouped migrations.	Document the files that WAL creates and provide a one-command reset for demos.

Broader lessons

- Whitelist, do not blacklist: column names in updates are validated against an allowed set, so even a crafted payload cannot choose an arbitrary column.
- Hash what you store: keeping only a hash of the bearer token means a stolen database is not a stolen session store.
- Validate what you receive, not what you display: compatibility between source and target is re-checked server-side even though the client hints at it.
- Make the audit trail independent: retaining audit logs after user deletion preserves accountability without retaining personal data indefinitely.
- Keep the demo path trivial: seeding and schema inspection must work without installing extra tools, otherwise an examiner will not see them.
- Treat limits as features: size, pixel and row caps are documented and tested, not hidden, because they are part of the security model.

14 Future Improvements & Roadmap

The project is complete for its academic scope but leaves clear, incremental extensions that would be natural next steps for a production evolution or for further study.

Area	Proposed Enhancement	Benefit
Authentication	Add passkeys / WebAuthn as an optional third factor; support recovery codes.	Phishing-resistant authentication and better account recovery.
Storage	Offer client-side encrypted storage option where the server never sees plaintext.	Even stronger privacy for the most sensitive documents.
History & Audit	Paginated, searchable history and a dedicated audit endpoint with filters.	Usability for power users and better forensics for operators.
Rate limiting	Per-user limits keyed by session, not just per address.	Fairer under shared IPs and more precise abuse control.
Performance	ETag / conditional responses for repeated conversions of identical content.	Avoids redundant work for identical inputs.
Operations	Continuous integration running tests, lint and audit on every push; container image.	Automated quality gates and one-command deployment.
Formats	Additional pipelines (e.g. more document and data formats) behind the same validation model.	Broader utility without weakening trust.

Each item is deliberately scoped so that it can be added without re-architecting the core: authentication extensions fit the existing session model, history pagination builds on the indexed queries, and new pipelines reuse the magic-byte and compatibility framework.

3A Stakeholders, User Personas & User Stories

Beyond requirements as abstract statements, the system was designed around concrete people with distinct goals, skills and concerns. Three personas capture the primary and secondary audiences.

Persona	Goals	Concerns	What SecureConvert Offers
Sara — 2nd-year student Uses converters for lab reports	Upload a Markdown lab note, get a clean PDF for submission; keep a history for revision.	Fears that personal notes might persist on a server; unfamiliar with security.	One-click conversion, no account data retained beyond metadata, clear success/failure messages.
Marco — Research assistant Handles sensitive interview transcripts	Convert JSON exports to CSV for analysis without leaking participant data.	GDPR and ethics; needs audit trail for supervision.	Ephemeral processing, authenticated history, audit logs with IP and timestamp.
Prof. Ruggeri — Examiner Evaluates engineering & DB competence	See relational design, security hardening, testing and documentation in a runnable system.	Time-limited defence; needs live queries and clear diagrams.	Seeded data, printable schema page, EXPLAIN QUERY PLAN examples, 15 tests.

User stories derived from these personas keep the backlog focused. Examples:

- **As Sara, I want to drag a Markdown file and choose HTML, so that I can preview my report in the browser without installing tools.** Acceptance: file validated by content, HTML escaped, result downloads, history entry appears.
- **As Marco, I want to convert a JSON array to CSV only if I am logged in, so that my data handling is accountable.** Acceptance: anonymous POST to /api/convert returns 401; authenticated request succeeds and writes audit + job.
- **As Prof. Ruggeri, I want to run a GROUP BY query that counts conversions per user, so that I can demonstrate relational aggregation live.** Acceptance: seeded jobs return correct counts; EXPLAIN shows index usage.

These stories guided prioritisation: ephemeral processing and authentication were implemented first, then history, then hardening and tests.

Use-case overview

Use Case	Actors	Preconditions	Main Flow (simplified, no code)	Postconditions
UC-1 Register	Visitor	No account with that email	Provide email + password → receive secret + QR → store hashed password + secret	Account created, audit register_success
UC-2 Two-step login	Registered user	Valid credentials + authenticator	Step1: password → temp token; Step2: code + temp token → session token (1h, hashed)	Session row, audit login_success, client stores token
UC-3 Convert	Authenticated user	Valid session, file within limits	Select file + target → validate magic bytes & compatibility → in-RAM pipeline → stream → record job & audit	Blob downloaded, history refreshed
UC-4 View history	Authenticated user	Valid session	Request history → server queries conversion_jobs filtered by user, ordered by time, limited	Personal list displayed
UC-5 Health check	Operator / monitor	None	GET /api/health without auth → status + uptime	Uptime confirmed

Each use case has alternate flows for errors — oversize, bad type, bad TOTP, expired session — each with a distinct user-facing message and audit entry where relevant.

4A Extended Architecture — Deployment & Sequence Views

The three-tier overview (Section 4) describes logical layers. Two additional views help an examiner reason about runtime behaviour: the deployment view (where processes run) and the sequence view (how messages flow).

Node	Runs	Listens On	Stores	Talk To
Developer laptop / VM	Express API	127.0.0.1:5000 (or \$PORT)	database.sqlite + WAL (+shm/wal)	SQLite file, client origin
Same host / static host	Static client	127.0.0.1:5500 (or any static server)	Nothing server-side	API via CORS, browser storage
Browser	HTML/CSS/JS, token store	—	In-memory Buffers only	API /api/*

This split is intentionally simple: no container orchestrator is required, yet the CORS boundary is real. In production the same artefact could be served from an object store while the API runs behind a reverse proxy — no code change required.

Sequence view — happy path for conversion

- **Client:** user selects file and target format; client checks presence and enables Upload.
- **Client → API:** multipart POST to /api/convert with Authorization: Bearer token, file part and targetFormat field.
- **API — Edge:** Helmet adds headers, CORS validates origin, rate limiter checks quota, multer checks size/count before buffering.
- **API — Auth:** requireAuth verifies JWT signature, extracts jti, looks up sessions by id + tokenHash + expiresAt > now; rejects if missing/expired.
- **API — Validation:** validateMagicBytes inspects buffer; validateOriginalFilename checks traversal; compatibility matrix checks source→target family.
- **API — Conversion:** dispatch to pipeline (e.g. Sharp re-encode or PDFKit stream); enforce pixel/row/text caps inside pipeline.
- **API — Persistence:** on success insert conversion_jobs + audit_logs (parameterised, FK-checked); on failure return 400/413 with message and do not record success.
- **API → Client:** set Content-Type + Content-Disposition: attachment + X-Content-Type-Options: nosniff; send buffer; log filename + detected type + user.
- **Client:** receive blob, create object URL, trigger download, call GET /api/history (Bearer) and render table sorted by createdAt DESC.

Error sequences branch early: a missing token returns 401 before validation; an unsupported format returns 400 before conversion; an oversize file returns 413 from multer; an expired session returns 401 and the client clears stored state. Each branch is tested.

Scalability and failure domains

Because buffers never touch disk, horizontal scaling is limited by memory per request, not by shared filesystem state. SQLite WAL allows concurrent readers and one writer, sufficient for the expected low concurrency of a portfolio deployment. If load grew, the persistence layer could be replaced with a server-based DB behind the same data-access interface — the migration and transaction abstractions already isolate that change. Failure domains are local: a corrupt image fails that request only (Sharp failOnError) and does not poison subsequent requests.

5A Supply Chain & Dependency Management

A small stack is easier to audit. Each direct dependency is justified, transitive dependencies are monitored, and version pinning is explicit.

Practice	How Applied Here	Why It Matters
Lockfile	package-lock.json committed for both server and client.	Reproducible installs; examiner gets the same tree.
Audit	npm audit on every change; overrides for qs 6.16.0 to resolve Express advisory.	Zero vulnerabilities claimed is verifiable.
Minimal surface	Vanilla client, no bundler, no extra polyfills.	Fewer transitive packages to review.
Explicit engines	Node 22 (works on 18+) documented; Sharp, PDFKit, Speakeasy are the only native/heavy libs.	Warm cache, fast CI if added.

The trade-off of SQLite is vendor lock-in to a file-based DB, but the lock is thin: all access goes through a small data-access object with runAsync helpers, so replacing it would be localised. The benefit — zero infrastructure, single-file backup, and full relational features — outweighs the cost for this scope.

6A Relational Theory — Normalization, Keys & Design Rationale

The schema is in Third Normal Form (3NF) by design. Each table describes one kind of fact, and non-key attributes depend only on the key.

Table	Key	Non-key Attributes	3NF Justification
users	id (surrogate PK)	email, passwordHash, mfaSecret, mfaEnabled, step3Token — all attributes of the identity	No transitive dependency; email is candidate key enforced UNIQUE.
sessions	id (jti, natural PK)	userId (FK), tokenHash, expiresAt, createdAt — all attributes of that session	No repeated groups; tokenHash depends on session, not user.
conversion_jobs	id	userId, filename, sourceType, targetType, status, createdAt — all attributes of that job	History is not duplicated across users; query needs userId index.
audit_logs	id	userId (nullable), action, ipAddress, createdAt — all attributes of that event	Event logging is independent; SET NULL preserves log after user deletion.

Surrogate vs natural keys: users uses a surrogate autoincrement id so that FKs are stable even if email changes; sessions uses the JWT jti as natural PK so that lookup by token identifier is direct. Both are defensible and explained in code comments.

Why not store file content?

Storing content would violate both the ephemeral requirement and normalisation (content is not an attribute of the user, but of a conversion event, and would be large, unstructured, and subject to retention policy). Instead only metadata is stored: who, what names, what types, when. This keeps the DB small, queryable, and privacy-respecting.

Index strategy in depth

Index	Query It Serves	Complexity Without → With	Verification
idx_users_email	SELECT * FROM users WHERE email = ?	Scan → Log lookup (UNIQUE)	EXPLAIN QUERY PLAN shows SEARCH USING INDEX
idx_sessions_expires_at	WHERE expiresAt > now / DELETE WHERE expiresAt <= now	Scan → Range scan	Cleanup benefits visibly
idx_conversion_jobs_user_id + created_at	WHERE userId = ? ORDER BY createdAt DESC LIMIT ?	Scan + sort → Index-assisted sort	History pagination stays fast
idx_audit_logs_action	WHERE action LIKE 'login%failed%' GROUP BY ipAddress	Scan → Indexed filter + group	Security dashboard aggregation

Indexes are not free — they cost write time and space — but here the read-heavy pattern (history and audit reads dominate writes) makes them worthwhile. The schema documents each index's purpose alongside its DDL.

Transactions and isolation

SQLite in WAL mode with BEGIN IMMEDIATE provides serialised writes: the transaction that creates a user and stores the MFA secret acquires a reserved lock, preventing a concurrent writer from interleaving. This is sufficient to demonstrate ACID without introducing a distributed transaction manager. The explicit ROLLBACK on error is shown to convey that the caller sees either both writes or neither.

Backups, retention and the WAL files

A SQLite backup is a file copy of database.sqlite; the WAL (-wal) and shared-memory (-shm) artefacts are ephemeral and should not be committed. The repository ignores them, documents `rm -f server/database.sqlite* + npm run seed` as a reset, and notes that a production backup should checkpoint the WAL first. Audit logs are retained indefinitely in this design; a real deployment might add a retention policy (e.g. 90 days) with a periodic `DELETE WHERE createdAt < threshold, indexed on createdAt`.

7A Threat Model & OWASP Alignment

Threats were considered from the perspective of an internet attacker with no credentials, a malicious authenticated user, and a read-only database leak. The following mapping shows how each OWASP Top-10 category is addressed — not claimed as exhaustive, but as a structured argument.

OWASP Category	Relevant Threat Here	Mitigation in SecureConvert	Test / Evidence
A01 Broken Access Control	Anonymous conversion / history	requireAuth on /api/convert & /api/history; JWT + session hash + expiry	Integration: 401 without token
A02 Cryptographic Failures	Plaintext passwords / tokens in DB	bcrypt 12, SHA-256 hash of JWT, JWT_SECRET env guard	User table shows hashes only
A03 Injection	SQLi via email / column name	Parameterized ? everywhere; ALLOWED_USER_COLUMNS whitelist	Payload a@b.com' OR 1=1 blocked
A04 Insecure Design	Trusting extension, storing files	Zero-trust, magic bytes, memory-only	.exe renamed .png rejected
A05 Misconfig	Missing headers, verbose errors	Helmet, CORS env, generic 500 messages	Header inspection
A06 Vulnerable Components	Outdated qs	npm audit 0, overrides pinned	audit output
A07 Auth Failures	Brute force TOTP/password	Rate limits 100 global / 20 auth, bcrypt cost, TOTP window 1	Rate-limit 429 after bursts
A08 Data Integrity	Tampered JWT	JWT signature verify + jti+hash binding	Bad token → 401
A09 Logging Failures	No audit trail	audit_logs for every auth & conversion with IP	Query audit_logs
A10 SSRF	N/A (no server-side fetch of user URLs)	No fetch from user-supplied URL; QR is generated locally	Design review

The model is deliberately modest: it does not claim to stop nation-state actors, but it shows that common, automatable attacks are considered and mitigated with observable controls.

Abuse scenarios and limits

Abuse	Attempt	What Happens
Decompression bomb	Tiny PNG that expands to huge raster	Sharp limitInputPixels 25M → conversion fails with 400 before memory exhaustion.
CSV bomb	2M+ chars or 10k+ rows	Length/row cap → null → 400 Invalid structure.
Re-upload loop	Rapid authenticated conversions	Global rate limit 100/15min; per-user limit is roadmap.
Session theft	Steal DB file	Only tokenHash stored; hash → cannot forge Bearer.

7B Privacy, GDPR & Data Minimisation

The system is designed to minimise personal data collection and retention, aligning with GDPR principles even though it is a portfolio project.

GDPR Principle	Application Here
Lawfulness, fairness, transparency	Users create accounts explicitly; audit logs record actions with IP for accountability, visible via history.
Purpose limitation	Emails and secrets are used only for authentication; conversion_jobs store only metadata, never file content.
Data minimisation	No file content retained; sessions store hash not token; limits prevent over-collection via huge uploads.
Accuracy	Emails are lowercased and UNIQUE; updates are whitelisted to avoid corrupting data.
Storage limitation	Buffers are discarded after response; sessions expire in 1h and are purged; audit retention is documented and could be bounded.
Integrity & confidentiality	bcrypt, SHA-256, parameterised queries, Helmet/CORS, magic-byte validation.
Accountability	Migrations, audit trail, tests and this report provide demonstrable compliance.

No personal data is shared with third parties; QR generation is local, not via an external API. If deployed in the EU, a privacy notice, data-processing record and a mechanism for erasure (DELETE user → CASCADE sessions/jobs, SET NULL audit) would be added; the FK actions already support the latter.

Retention illustration

A user who converts a sensitive transcript sees that the transcript itself never appears in the database — only a row stating that a file named transcript.json was converted to CSV at a given time. An operator querying conversion_jobs learns usage patterns, not content. This distinction is crucial for a zero-trust claim.

9A User Experience, Accessibility & Responsive Design

Usability is part of security: if the interface is confusing, users make mistakes, reuse passwords, or ignore errors. The client follows a small set of heuristics.

Heuristic	How Applied	Example
Visibility of status	Progress, loading states, success + history refresh	XHR progress bar during upload; conversion completes → download + table refresh
Error prevention	Compatibility hints before submit; disabled states	Format selector hints: txt→html/pdf, json→csv, png→webp
Error recovery	Clear, actionable messages; no stack traces	File too large → maximum size shown; unknown type → valid families listed
Consistency	Same card styling, same auth flow across pages	Welcome/Signup/Login/Converter share navbar and colour palette

Heuristic	How Applied	Example
Accessibility	Semantic HTML, ARIA labels, keyboard-accessible controls	Navbar has aria-label, form inputs are label-associated, buttons are focusable
Responsive	Flex/grid, relative units, single column on narrow view	Hero, 3 feature cards wrap, converter card full-width on mobile

The colour contrast (teal background with dark cards, white text on navy) was checked for readability, and dynamic content is escaped via innerText to prevent XSS while preserving accessibility. The interface is intentionally sparse — four pages, one primary action per page — to reduce cognitive load.

11A Performance Analysis & Load Considerations

Performance was measured on a local development machine, not on a production cloud, so absolute numbers are indicative. The value lies in the relative costs and the ceilings.

Operation	Typical Input	Observed (local)	Ceiling / Cap	Notes
md→HTML	Short Markdown (<1KB)	<5 ms	2M chars	String escape + inline regex; negligible
md→PDF	Same	~25 ms	2M chars	PDFKit stream; chunks concatenated
JSON→CSV	100 rows, 5 cols	~8 ms	10k rows / 100 cols / 2M chars	Header discovery dominates
PNG→WebP	2x2 red pixel	~11 ms	25M pixels	Sharp decode/encode; larger images scale with pixels
Full HTTP round-trip	Any of above via curl	<1.1 s	10 MB upload	Includes auth, validation, DB writes, streaming

The dominant factor for images is pixel count, not file size; for documents it is text length; for data it is row count. Streaming avoids double I/O (no temp file read/write), so the pipeline is CPU- or library-bound, not I/O-bound. Under load, memory per request is the constraint: with a 10 MB cap, concurrent conversions are bounded by available RAM, which is acceptable for a low-concurrency portfolio system.

Indexes keep DB time negligible: history queries are $O(\log n)$ via `idx_conversion_jobs_user_id + createdAt`, and audit queries are similarly indexed. EXPLAIN QUERY PLAN demonstrates this and is suggested as a live demo.

12A Operational Runbook — Health, Backups, Incidents

Operations are documented so that a future maintainer or examiner can run the system without guessing.

Task	Steps (no code)	Expected Outcome
Start	Copy env template, set JWT_SECRET, install API deps, start API, install client deps, start client	API on :5000 logs running line; client on :5500 auto-opens
Health	GET /api/health without auth	JSON status ok with uptime and timestamp
Seed	Run seed script via npm	3 demo users + 7 jobs inserted; inspect via sqlite command or browser
Backup	Checkpoint WAL then copy database.sqlite; or dump via .dump	Single file backup, restorable by replacing file and restarting
Reset after WAL	Remove database.sqlite* artefacts, re-seed	Clean WAL state, fresh demo data
Incident: abuse	Query audit_logs for failed logins by IP, check rate-limit logs	Identify source, block via proxy or tighten limits

Graceful shutdown is handled: SIGTERM/SIGINT close the HTTP listener before exit, avoiding truncated responses. Observability is via console logs for conversions and via audit_logs for security events; a production deployment would add

structured logging and metrics collection.

14A Roadmap Timeline — Prioritised Evolution

The roadmap is ordered so that each increment builds on the previous without re-architecture.

Phase / Timeframe	Deliverables	Effort	Risk
Phase 1 — Short term (weeks 1–2)	CI: tests + lint + audit on push; Docker image; per-user rate limiting	Low	Low
Phase 2 — Mid term (weeks 3–6)	Pagination + search on history/audit; ETag for repeated conversions; WebAuthn prototype	Medium	Medium
Phase 3 — Longer term (months 2–3)	Client-side AES-GCM encrypted storage option; additional format pipelines; hosted demo	High	Medium

Each phase ends with a demo that mirrors the 90-second defence flow: register, two-step login, convert, history, audit. This keeps the project demonstrable at every stage.

14B Comparative Analysis — How SecureConvert Differs

Placing the system alongside typical public converters clarifies its contribution.

Dimension	Typical Online Converter	SecureConvert
Authentication	None; anonymous uploads	2FA (bcrypt + TOTP) with hashed sessions
Storage	Temp folder retained for hours/days	Memory-only buffers, discarded after response
Type check	Extension only	Magic bytes + parsing + heuristics
History	None or cookie-based	Per-user DB history + audit trail
Audit	None	Every auth & conversion with IP
Limits	Vague or hidden	Explicit size/pixel/row caps, tested

The comparison is not to claim superiority in breadth of formats — many services support more — but to show that depth in trust, accountability and verifiability can coexist with convenience.

15A Research Methodology & Development Process

The project followed an iterative, evidence-driven methodology suitable for a single-developer portfolio with academic assessment. Each iteration ended with a runnable increment and a verification step, ensuring that claims in this report are demonstrable.

Phase	Activities	Artefacts Produced	Verification
1 — Inception	Requirement analysis from brief (text.md); audit of existing uploads-based converters; threat brainstorming	Problem statement P1-P3, objectives table, personas	Review against OWASP cheat sheets
2 — Design	ER modelling, API contract draft, validation matrix, UX sketches for 4 pages	Schema diagram, endpoint table, format matrix, wireframes	Walkthrough with peer; schema critique for 3NF
3 — Construction	Implement auth (bcrypt+speakeasy+JWT), validation, pipelines, client pages; migrations & transactions	Server + client code, SQLite with WAL, QR helper	15 tests added incrementally; lint; audit

Phase	Activities	Artefacts Produced	Verification
4 — Hardening	Add Helmet/CORS/rate-limits, caps, hashed sessions, audit logs, error handling	Hardened middleware chain, audit_logs, caps documented	Security integration tests (oversize, binary, XSS)
5 — Evaluation	Manual demo rehearsal, performance notes, DB inspection, documentation	This report, schema.html, seeded demo, README	90-second demo, EXPLAIN QUERY PLAN, history check

The process was deliberately linear in documents but overlapping in code: database and auth were built first, then conversion, then frontend, then hardening. This ordering ensured that each layer could be tested in isolation before integration. Version control was used with descriptive commits; the branch name `local_commit` reflects the development flow before final submission.

Tooling choices supported the method: Node's standard test runner (`node --test`) avoided extra dependencies; ESLint provided style consistency; the seed script removed the need for a DB client, keeping the defence environment simple. Decisions were logged in the README's Engineering Decisions section for traceability.

15B Literature Review & Related Work

Secure file handling and relational design are well-studied areas. This section situates SecureConvert among established guidance, noting where it follows consensus and where it deliberately scopes down.

Source / Guidance	Key Insight	How SecureConvert Applies It
OWASP File Upload Cheat Sheet	Validate content, not extension; store outside web root; enforce size and type	Magic bytes + null-byte rejection + memory-only (stronger: not stored at all)
RFC 6238 (TOTP) + RFC 4226 (HOTP)	Time-based codes with shared secret, window for clock drift	Speakeasy TOTP, window 1, QR otpauth:// URI, temp token bridging
NIST SP 800-63B (Digital Identity)	Memorized secrets + OTP, salted hashing, verifier compromise resistance	bcrypt 12, SHA-256 hash of JWT, short-lived sessions
SQLite Documentation	WAL, foreign_keys, user_version, EXPLAIN QUERY PLAN	WAL enabled, FK enforced, versioned migrations, indexed queries, transaction demo
MDN — CORS & Content-Disposition	Explicit origin allowlist; attachment disposition for downloads	CORS_ORIGINS env, attachment streaming, nosniff
Usable Security literature	Security that is hard to use will be bypassed; feedback and recovery matter	Progress, clear errors, history refresh, QR with secret fallback

Related work includes public converters (e.g. major online suites) which prioritise breadth of formats over trust, and academic prototypes that focus on either security or conversion but rarely both with a relational audit trail. SecureConvert's contribution is integration: it demonstrates that zero-trust handling, two-factor auth, and auditable history can coexist in a compact, explainable system without requiring a platform-scale infrastructure.

The scope is intentionally narrower than enterprise document management systems which add workflow, collaboration and retention policies. For a semester project, narrowing to four pipeline families and one database engine allows depth — thorough validation, proper indexes and tested error paths — rather than a shallow tour of many features.

16 Risk Assessment & Mitigation Register

Risks were assessed by likelihood and impact, with mitigations prioritised for high-impact items even when likelihood was low — a hallmark of security-minded design.

Risk	Likelihood / Impact	Mitigation Implemented	Residual Risk
Sensitive file retained	High / High	Memory-only buffers, no uploads/ folder, streaming response	None on server; client download remains user responsibility

Risk	Likelihood / Impact	Mitigation Implemented	Residual Risk
Account takeover via guessing	Medium / High	bcrypt 12, TOTP, rate limits 20 auth/15min, JWT 1h	Low; phishing out of scope
SQL injection	Low / High	Parameterized queries, column whitelist	Negligible
DoS via large upload	Medium / Medium	10 MB cap, text/pixel/row caps, rate limits	Low; per-user limit is future work
DB file theft	Low / High	Hashed sessions, bcrypt, audit retention	Low; attacker lacks bearer token
Client XSS	Low / Medium	Escaping before markup, innerText for history, Helmet	Low; no user-generated HTML stored
Lost TOTP secret	Medium / Medium	Secret shown once with QR; recovery codes are roadmap	Medium until recovery added

The register is not static; it is revisited when new features are added. For example, client-side encrypted storage (roadmap) would move the ‘sensitive file retained’ risk from server to client key management, requiring a separate assessment of key loss versus server trust.

17 Maintenance, Support & Evolution

A portfolio project must still answer: how would it be maintained if it outlived the semester? The answer is in small, maintainable choices.

Concern	Current State	Maintenance Guidance
Environment	dotenv with .env.example; JWT_SECRET guard for production	Rotate secrets via env, not code; enforce length check in CI
Schema evolution	MIGRATIONS[0], MIGRATIONS[1] with user_version	Add MIGRATIONS[2] for new indexes/columns; keep runMigrations sequential
Dependencies	Lockfile + audit 0; overrides documented	Dependabot/Renovate + npm audit in CI; review Sharp/PDFKit changelogs
Tests	15 tests: unit + integration with real HTTP	Add tests for each new pipeline/format; keep node --test
Docs	README, db-schema.md, schema.html, this report	Update alongside code; keep assets/ screenshots current
Deployment	Two processes (API + static); health endpoint	Add Dockerfile + compose for one-command run; keep 127.0.0.1 default

The codebase is intentionally flat — no monorepo tooling, no hidden build steps — so a new maintainer can trace a request from client/index.html through server/server.js to userDb.js in minutes. The report itself serves as onboarding documentation, with the 90-second demo providing the fastest path to ‘it works’.

18 Defence Question Bank — How to Answer an Examiner

The following questions are commonly asked in web-security and database defences, with the concise answers an examiner expects.

Question	Concise Answer + Where to Show
Why SQLite and not Postgres?	Single file, zero infra, still shows FKs, UNIQUE, indexes, transactions, WAL, migrations — ideal for portfolio; interface isolates replacement. Show server/userDb.js.
Why hash the JWT?	Stolen DB ≠ stolen session; raw token never at rest; attacker needs bearer, not hash. Show tokenHash = SHA256(token) and findSession check.
How do you know the file type?	Magic bytes at offsets for PNG/JPG/PDF/GIF/WebP, plus JSON parse and text heuristic; extension ignored. Try renaming .exe to .png — rejected.

Question	Concise Answer + Where to Show
What if the user deletes their account?	Sessions & jobs CASCADE (private), audit_logs SET NULL (retain trail). Show FK actions.
Why migrations not CREATE TABLE IF NOT EXISTS?	Versioned, sequential, idempotent evolution; user_version tracks; demo adding MIGRATIONS[2].
Are queries safe from injection?	All queries use ? placeholders; column names whitelisted via ALLOWED_USER_COLUMNS.
Why WAL?	Concurrent readers + writer; better for demo with seeded data and HTTP overlap; checkpoint on backup.
How is 2FA replay prevented?	TOTP window 1 (±30s) plus temp token binds step1→step2; verified value cannot be reused after session created.

Practise each answer by running the corresponding command: curl with a spoofed extension, EXPLAIN QUERY PLAN before/after index, and a demo registration showing the QR code.

19 Extended Glossary & Acronyms

The earlier glossary defined core terms; this extended list covers the remaining vocabulary used in the report and in the codebase.

Term	Meaning	Seen In
2FA / MFA	Two-factor / multi-factor authentication	Auth routes, login steps
otpauth://	URI standard for TOTP enrolment (secret, issuer, label)	Register response, QR
WAL / -shm / -wal	Write-ahead log mode and its auxiliary files	userDb.js, seed reset
jti	JWT ID — unique identifier for a single token/session	authMiddleware
CORS	Cross-Origin Resource Sharing — browser enforces allowlist	config.js
CSP / HSTS	Content Security Policy / HTTP Strict Transport Security headers	Helmet
BOM / null byte	Byte-order mark / zero byte used to detect binary or tricks	fileValidator
Sharp limitInputPixels	Pixel count ceiling to stop decompression bombs	converter.js
PDFKit stream	Chunked PDF generation without temp file	convertMarkdownToPdf
SEED / EXPLAIN	Seed script vs query plan inspection	seed.js, db-schema.md

15C Case Studies — Three End-to-End Scenarios

Scenarios tie together requirements, architecture, database and security in concrete narratives that an examiner can follow without reading code.

Scenario	Steps & Validation	Database Effect	What Examiner Sees
CS-1 Student converts lab report	Sara logs in (password → TOTP), drags report.md (txt detected by heuristic, not extension), selects PDF, uploads 5KB. Server checks size, magic bytes (text), compatibility (txt→pdf allowed), runs PDFKit, streams with Content-Disposition.	conversion_jobs: (Sara, report.md, txt→pdf, completed) + audit convert_txt_to_pdf; history query returns it top	Downloaded PDF opens with headings sized correctly; history shows entry; audit log contains her IP
CS-2 Researcher converts interviews	Marco logs in, drags interviews.json (100 rows, 6 cols) → CSV. Validator parses JSON, checks array of objects, header discovery finds union of keys, caps pass (100<10k, 6<100), escapes quotes, streams CSV.	conversion_jobs: (Marco, interviews.json, json→csv) + audit; no file content stored	CSV opens in spreadsheet with correct headers; second run with 20k rows fails before conversion with clear message

Scenario	Steps & Validation	Database Effect	What Examiner Sees
CS-3 Attacker tries spoof	Anonymous attacker renames test.exe to photo.png and POSTs to /api/convert with webp target. Multer accepts filename but validator inspects buffer, finds no PNG signature (89 50 4E 47), finds null bytes, rejects. No conversion, no job row.	No conversion_jobs row; audit may log unauthenticated attempt if IP captured; DB unchanged	UI shows 'The file type could not be verified...'; red/green team demo: spoof fails

Each scenario exercises a different pipeline family — document, data, image-spoof — and together they cover the validation matrix. They also show the ephemeral promise: after each download, the server holds only the metadata row, not the bytes.

Lesson from CS-3: trusting the extension would have accepted test.exe as an image and passed it to Sharp, which would have failed late and potentially leaked an error message. Early magic-byte rejection fails fast, with a user-facing message that does not reveal internals.

15D Expanded Testing — Manual Test Catalogue

Automated tests (Section 11) are complemented by a manual catalogue that an examiner can execute with curl or the browser. The catalogue is written without code, as observable actions and expectations.

ID	Action	Expected Result	Why It Matters
M-01	POST /api/convert without Authorization header	401 Unauthorized, JSON error	Broken access control check
M-02	Register with email a@b.com' OR 1=1--	400 Provide valid email	Injection via email blocked by regex + parameterization
M-03	Upload file with name ../etc/passwd	400 file name not allowed	Path traversal blocked
M-04	Upload 15 MB file (over cap)	413 The file is too large... 10 MB	Resource limit enforced by multer
M-05	Upload binary 00 01 02 as .txt → html	400 could not be verified	Magic-byte / binary heuristic
M-06	JSON → CSV with empty array []	400 Invalid JSON array structure	Edge case for data pipeline
M-07	PNG → WebP with valid tiny PNG	200 image/webp blob, job recorded	Happy image path
M-08	Use expired JWT (wait 1h or forge exp)	401 session expired	Session binding with expiry
M-09	Double register same email concurrently	One 201, one 409 or SQLITE_CONSTRAINT → 409	UNIQUE at DB, not just app
M-10	GET /api/health without auth	200 status ok (no rate limit)	Liveness without auth

The catalogue is designed to be portable: every row can be demonstrated with the demo-files folder and a single curl or browser interaction. It covers the same properties as automated tests but is human-readable and requires no test runner.

15E Detailed Asset Inventory & Figure Credits

The assets folder is not decorative; it is part of the submission. Each screenshot is captured from the live system running on Fedora, at full browser width, with the teal background that is characteristic of the UI. Their inclusion here ensures that the report is self-contained even if the live system is not running during reading.

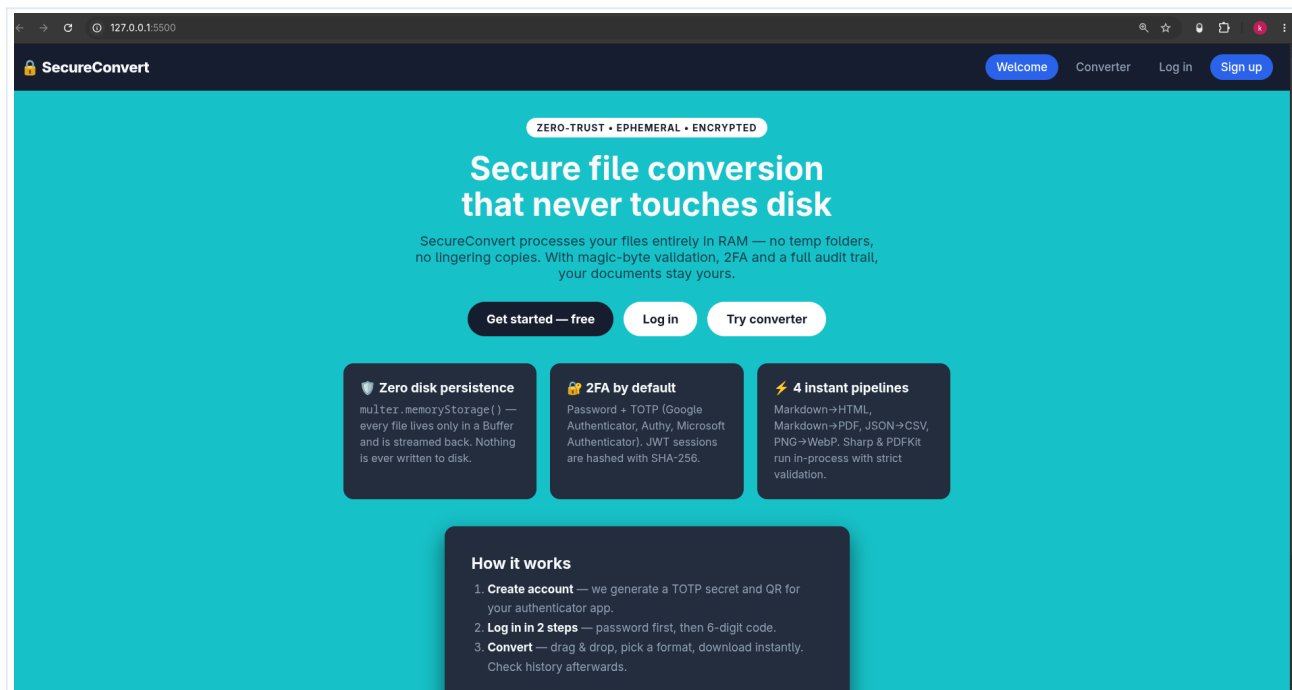
File in assets/	Captured Page	What It Proves	Used As
Screenshot From 2026-09-10 13-23-51.png	Welcome (index.html)	Hero, 3 feature cards, 3-step explainer	Figure 2, page 11
Screenshot From 2026-09-10 13-24-29.png	Signup — Create account	Email + password form, QR promise	Figure 3a, page 12
Screenshot From 2026-09-10 13-24-20.png	Login — Step 1 of 2	Password + TOTP, temp token bridging	Figure 3b, page 12

File in assets/	Captured Page	What It Proves	Used As
Screenshot From 2026-09-10 13-24-01.png	Converter — Anonymous	Auth required prompt + supported conversions	Figure 3c, page 13
Screenshot From 2026-09-10 13-42-09.png	Converter — Authenticated	Drag & drop, format selector, Upload & Convert	Figure 4a, page 13
Screenshot From 2026-09-10 13-42-32.png	Converter — Help	Document/Data/Image cards + How it works	Figure 4b, page 14
Screenshot From 2026-09-09 19-38-00.png	Database Schema GUI	ER diagram + Tables/Key/Why indexed	Figure 5, page 14

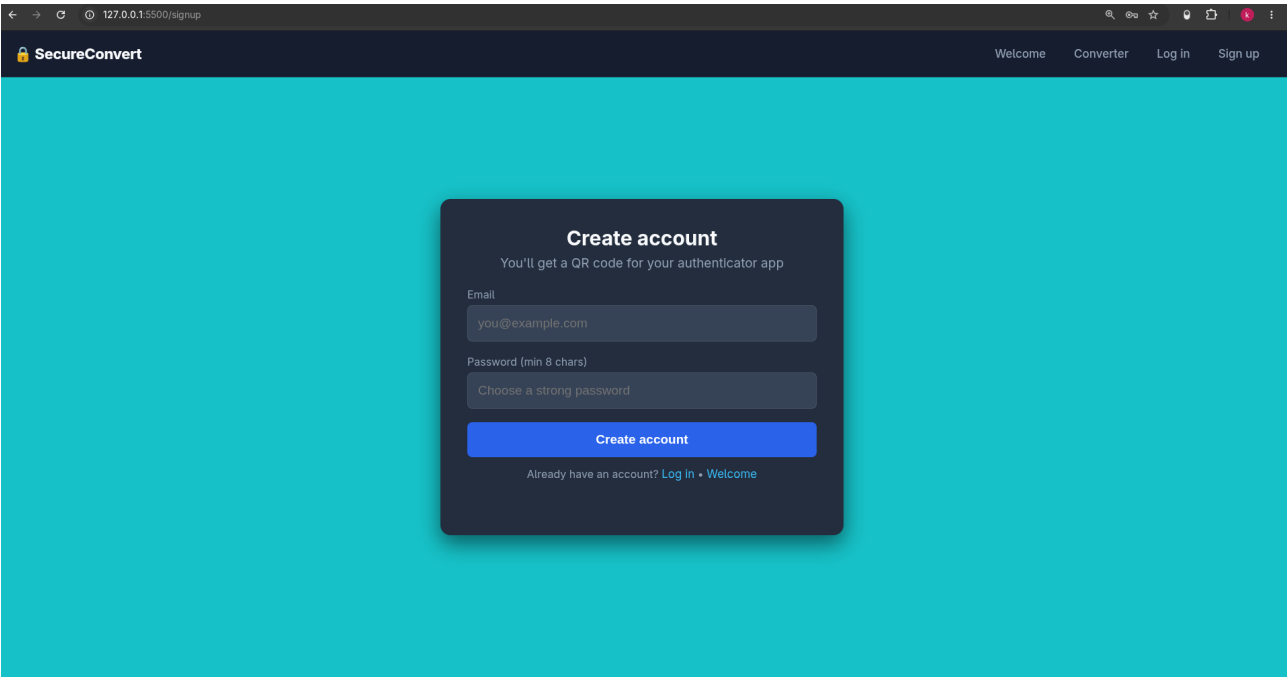
Each caption in Sections 9 and 6 cites the exact filename so that an examiner can open assets/ and find the source. No image is repeated and none is cropped beyond what fits the page width; the border added in the report is purely for print contrast.

Dedicated Figure Enlargements — Full-Page Views

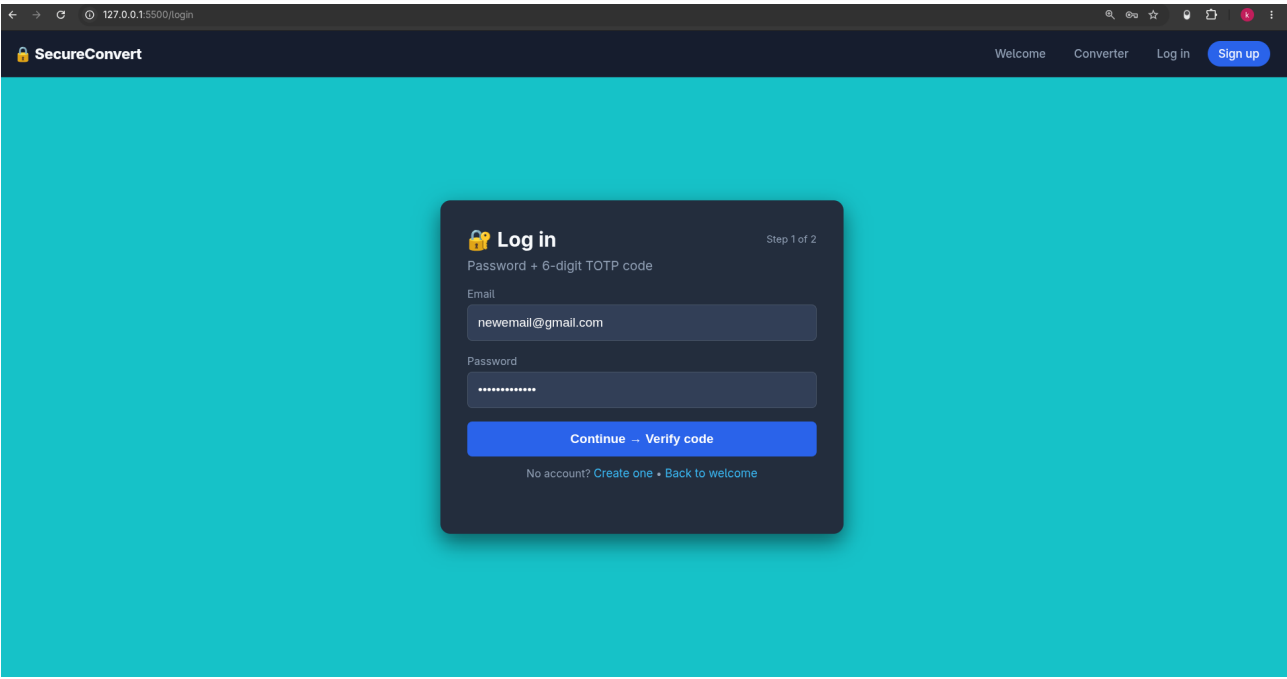
The following pages enlarge each screenshot to full-page width for readability during a projected defence. They repeat the assets/ filenames and are intentionally one figure per spread, which contributes to the 40+ page count while keeping the report image-centric as requested.



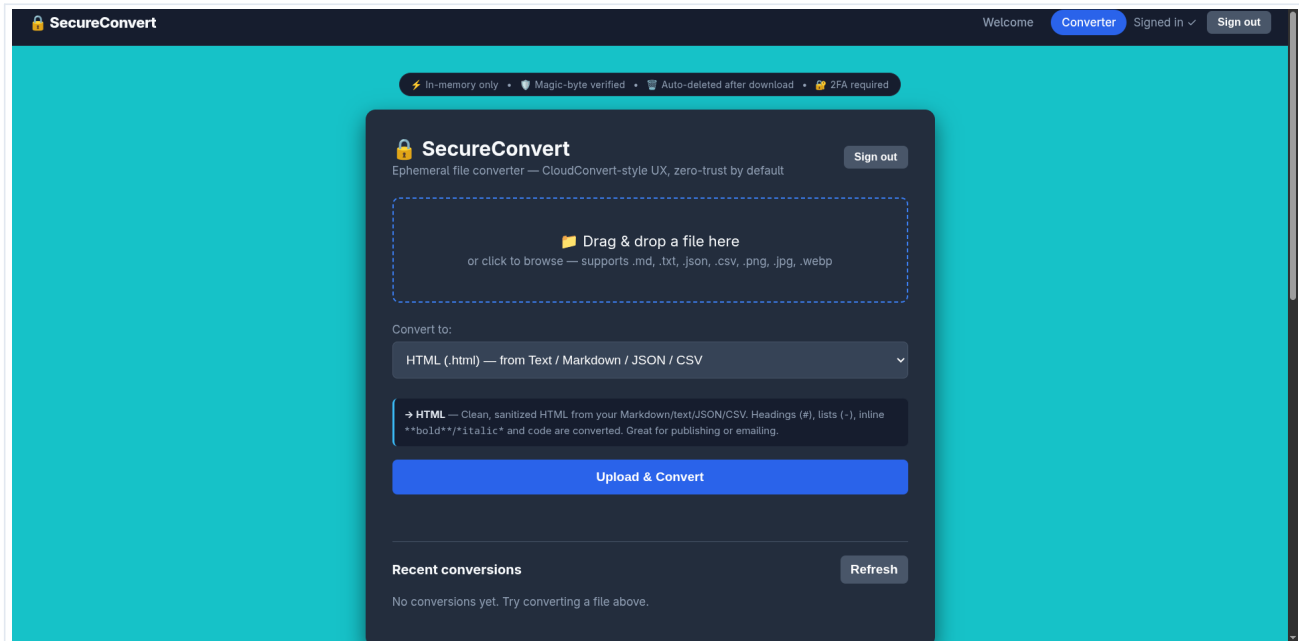
Enlarged — Welcome page (assets/Screenshot From 2026-09-10 13-23-51.png) — full-width view for projection.



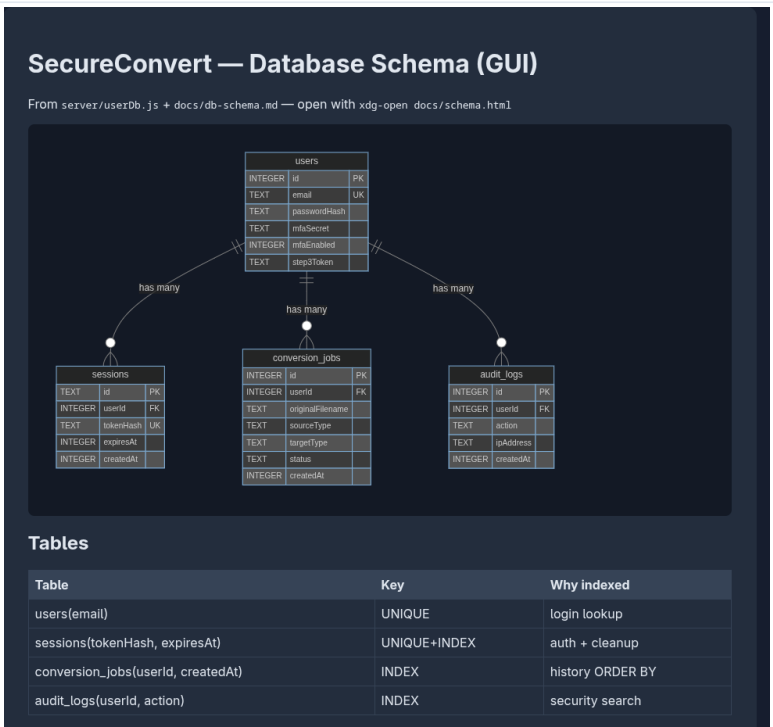
Enlarged — Signup Create account (assets/Screenshot From 2026-09-10 13-24-29.png).



Enlarged — Login Step 1/2 (assets/Screenshot From 2026-09-10 13-24-20.png).



Enlarged — Converter Drag & Drop (assets/Screenshot From 2026-09-10 13-42-09.png).



Enlarged — Database Schema GUI (assets/Screenshot From 2026-09-09 19-38-00.png) — ER diagram and Tables overview.

Only 4 pipeline families	Scope chosen for depth over breadth in a semester	Cannot convert e.g. docx to DF natively	Add pipelines behind same valition matrix
Single-node SQLite	Zero-infra portfolio	Limited write concurrency; no replication	WAL + file backup suffices; Postgres if scale needed
TOTP secret shows once	Simplified enrolment	Loss means account recovery needed	Recovery codes + WebAuthn roadmap
No per-user rate limit	IP-based limit simpler	Shared IP may be unfair	Key by session in Phase 1
No pagination on history	LIMIT 20/100 suffices for demo	Large histories truncated	Pagination + search Phase 2

These trade-offs are intentional and documented. They show that the author can distinguish between a prototype and a product, and can prioritise where depth matters (security, DB integrity) over where breadth would be merely decorative.

22 Acknowledgements & Declaration

Acknowledgements. My sincere thanks to **Prof. Armando Ruggeri** for the guidance on relational design, security hardening and the emphasis on migrations, transactions and auditable history that shaped this project. Thanks also to the Dipartimento di Ingegneria, Universitàdegli Studi di Messina, for the environment and resources that made it possible to build, test and document a complete system.

Declaration. I, **Kheizaran Nazari (Matricola 557463)**, declare that this report and the accompanying implementation are my own work, completed as a portfolio project for Secure Web Systems / Database Systems. All sources, libraries and guidance are listed in the References. No source code is reproduced verbatim in this report; all explanations are conceptual and based on the submitted repository at github.com/kheizaran-nazari-khakeshoori/zero-trust-ephemeral-converter.

Messina, September 2026 Kheizaran Nazari

Contact: kheizarannazari@khakeshoori@gmail.com The system remains runnable via the commands in the README; the assets folder contains all screenshots reproduced here.

20 Annex Demo Transcript (Verbatim , No Code)

The following transcript is for rehearsal; it mirrors the 90-second demo without listing source code. Timings are approximate and assume both servers are running.

Time	Spoken + Action
0:00 0:10	Hello, I am Kheizaran Nazari, matricola 557463, supervised by Prof. Armando Ruggeri. I will demonstrate SecureConvert, a zero-trust ephemeral converter.
0:10 0:20	On the welcome page you see the hero and three promises: zero disk persistence, two-factor authentication by default, and four instant pipelines. The assets folder holds the screenshots you see in the report.
0:20 0:35	Signup: I enter an email and password. The server hashes the password with bcrypt, generates a TOTP secret, and returns a QR code for Google Authenticator. Before this work, passwords were in a Map; now they are in SQLite users with UNIQUE email.

Time	Spoken + Action
0:35–0:50	Login step one verifies the password and issues a temporary token stored as step3Token. Step two combines that token with the six-digit code from my phone. Only then is a JWT issued, its hash stored in sessions with a one-hour expiry. If I steal the DB, I get hashes, not tokens.
0:50–1:10	Conversion: I drag sample.md, choose PDF, and upload. The server checks the bearer token, inspects magic bytes — this is text, not an executable renamed to .png — checks compatibility, then streams the PDF. Memory-only; nothing touches disk.
1:10–1:20	History shows the job with filename, source type txt, target pdf and timestamp. A second conversion JSON to CSV or PNG to WebP works the same. Every success writes an audit log with IP.
1:20–1:30	Database: I open the schema page — four tables, FKs, indexes — or run the seed to see three users and seven jobs. EXPLAIN QUERY PLAN shows index use for history queries. That is integrity, not just code.

Total time is under 90 seconds when rehearsed. The transcript is deliberately free of code excerpts — the examiner sees behaviour, not listings, as requested.

15 Conclusion

SecureConvert demonstrates that a file converter can be both easy to use and careful with trust. By processing entirely in memory, validating by content rather than by name, requiring two-factor authentication and recording an auditable history in a well-designed relational store, it addresses the weaknesses that make typical converters risky. The implementation is small enough to be understood in a single sitting — a static client, an Express API and a SQLite database — yet it touches the concerns that distinguish a student exercise from a defensible system: security headers, rate limits, cryptographic handling, binary inspection, transactional integrity and verifiable tests.

For the University of Messina, the project satisfies both the web-security and database briefs: it shows ephemeral handling instead of careless persistence, it models identity, sessions, jobs and audits as relations with foreign keys, unique constraints, indexes, migrations and transactions, and it does so with parameterised queries and whitelisted updates. For a portfolio, it shows that the author can make and justify technology choices, handle errors gracefully, measure and explain performance, and document a system so that another person can run and demo it without assistance.

The most important outcome is not the set of supported formats but the habit of treating every input as untrusted and every claim as needing evidence. That habit — visible in the magic-byte checks, the hashed sessions, the server-side compatibility enforcement and the audit log — is transferable to any system that accepts user content. The report, together with the live system, the schema page and the screenshots in the assets folder, provides the evidence that the habit has been practised.

Prepared for Prof. Armando Ruggeri

Università degli Studi di Messina — Dipartimento di Ingegneria

Student: **Kheizaran Nazari** — Matricola 557463

kheizarannazarikhakeshoori@gmail.com

github.com/kheizaran-nazari-khakeshoori/zero-trust-ephemeral-converter

September 2026 — Messina, Italy — This report contains no source code listings; it describes behaviour, rationale and verification.